Adevar Labs

Prepared for:

Neutral Trade

October 03, 2026

Neutral Trade Audit Report

Version: after fix review

Table of Contents

1. Executive Summary 5

About Neutral Trade 5

Audit Summary 5

Overall Security Posture 5

Before Fix Review Summary 6

After Fix Review Summary 6

Launch Recommendations 6

Audit Scope 7

2. Assumptions and Considerations 8

Audit Assumptions 8

3. Severity Definitions 9

Impact 9

Likelihood 9

Severity Classification Matrix 9

4. Findings 11

H01: Closing the switch-target account after request_bundle_switch leaves a withdrawal nothing can settle, so the source bundle can never unpause 11

M01: Fee rate changes apply to elapsed time and accrued gain, so raising a rate charges holders for a closed window 13

M02: Oracle refresh requires the pause that the tokenized path forbids, halting tokenized deposits between netting cycles 16

M03: Deposit crediting zero shares increments the pending counter and permanently freezes the bundle 19

L01: At a zero performance fee the high-water mark never commits, so a later rate rise prices gain against a stale mark 22

L02: The withdrawal curve is unvalidated on both bundle write paths, while the per-user path guards it 24

© 2026 Adevar Labs. All rights reserved.

L03: Manager reprices a committed withdrawal by raising the fee rate before settlement 26

L04: Bundles where the manager and keeper share one key freeze permanently if that key stops 28

L05: Per-user withdrawal-fee override applies to any receipt token its holder redeems, so exits route around the treasury fee 30

L06: Instant mint never credits the referrer whose book the tokenized exit later debits in full 32

L07: Withdrawal fee transfer to the treasury has no zero-amount guard, so a dead treasury halts every redemption 34

L08: Zero total assets reads as bootstrap, so the next deposit mints 1:1 against shares already outstanding 36

L09: Pool watermark nets out the performance fee but not the management fee, so every management charge fakes a loss 38

L10: Deposit watermark blend is gated on the live share price, so shares issued at the netting price keep the wrong watermark 41

L11: Deactivating a referrer freezes their referred total instead of letting withdrawals reduce it 43

L12: No instruction retires an unpayable withdrawal, so recovery depends on an operator workaround 45

L13: Pool loss carry does not scale on exit, so remaining token holders inherit an untaxed fee shield 47

L14: apply_fees_to_user charges fees against an oracle it never checks for staleness 50

L15: Receiver resets the oracle freshness clock and removes the equity buffer through perform_refill 53

L16: Switch falling back to the user wallet pays the discounted switch fee instead of the source withdrawal fee 56

5. Enhancement Opportunities 59

E01: A strategy receiver cannot be replaced in place, and disabling one also blocks the refill path 59

E02: Manager fee withdrawal has no distribution gate, so it spends cash a started distribution already counted 61

E03: The issuance fee's effective rate scales with pool size, so the configured rate is not the rate a depositor bears 63

E04: Instant mint bounds slippage on token count instead of value, so a tighter bound is cheaper to grief 65

E05: The withdrawal redemption schedule is checked only for non-zero values, so a transposed or expired schedule goes unnoticed 67

© 2026 Adevar Labs. All rights reserved.

E06: Strategy records no per-receiver debt, so one receiver's refill consumes another's repayment headroom 69

About Us 71

About Adevar Labs 71

Audit Methodology 71

Confidentiality Notice 73

Legal Disclaimer 73

© 2026 Adevar Labs. All rights reserved.

1. Executive Summary

About Neutral Trade

Neutral Trade is a Solana protocol that pools depositor funds into managed bundles. The **ntbundle** program issues each depositor a share of a bundle and accounts for deposits, withdrawals, switches between bundles, fees and referrals. A manager sets each bundle's parameters, and a keeper runs the netting, oracle and settlement cycle. Bundle funds are allocated to external strategy receivers, and a bundle can also mint transferable receipt tokens against pooled shares.

Audit Summary

The table below summarizes identified vulnerabilities by severity and remediation status.

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	1	1	0
Medium	3	3	0
Low	16	4	12

Enhancement opportunities: 6.

Overall Security Posture

The team fixed every High and Medium finding, fixed four Low findings, and acknowledged the rest. We reviewed eleven remediation pull requests across three delivery rounds, verified each fix against the code it changes, and ran the full test suite at the fix tip, where all 121 tests pass. The program's arithmetic remains well guarded, and rounding favors protocol solvency over protocol revenue. The residual risk now sits in operations rather than code: the keeper decides when to refresh the oracle, when to charge fees and which accounts to pass. Resolving the oracle deadlock also removed the staleness bound on the paths that price the tokenized pool, so the keeper now owns that timing.

© 2026 Adevar Labs. All rights reserved.

Before Fix Review Summary

The initial assessment found one High-severity, three Medium-severity and 16 Low-severity issues, plus six enhancement opportunities. The most serious findings concerned liveness. One High, one Medium and one Low shared a single shape: a request enters a queue, a counter increments, and only a successful settlement decrements it. A user who closed their own account therefore blocked settlement for the whole bundle. A second Medium halted tokenized activity between cycles because refreshing the oracle requires a pause that minting and redeeming forbid. The remaining findings concerned withdrawal fee repricing and bypass, referral tracking, fee and watermark accounting, oracle freshness and missing input validation.

After Fix Review Summary

A zero-share netting batch is now refunded in full instead of freezing the bundle, the switch instruction settles to the user's wallet when the target account is gone, and the keeper now creates the payout token account before refunds and withdrawals. The tokenized pool's fee baseline now releases its pro-rata slice when management-fee shares leave it, so the pool's watermark per share behaves like a classic high-water mark. The oracle deadlock is resolved.

Each fix review note records the team's position in every case where a residual risk rests on operational controls, and we agree with that disposition. Four items still need action before a bundle is tokenized, and they appear under Launch Recommendations.

Launch Recommendations

Mandatory

- Extend the keeper's idempotent token-account creation to **process_switch** and **process_referrer_withdrawal**.
- Set a non-zero **issuance_fee_bps** on every bundle before tokenizing it.
- Set a real **oracle_max_age** on the six bundles running zero and the one running roughly 1,000 days.
- Set a non-zero **mint_cap_per_cycle** on every bundle before tokenizing it.

Strongly recommended

- Fall back to any other token account the user owns when the owner authority has been reassigned.
- Document the charging window for **apply_fees_to_tokenized_depositor**.
- Extend the pool loss-carry disclosure to receipt-token transfers.
- Move the five bundles where the manager and keeper share one key onto separate keys with **set_keeper**.
- Bound **oracle_buffer** below 100 in both **set_oracle_buffer** and **initialize_bundle**.
- Refresh the redemption schedules on live bundles.
- Alert when a pending counter stays non-zero across two cycle attempts, when an oracle write repeats the previous value, and when a fee charge prices off an oracle older than **oracle_max_age**.

© 2026 Adevar Labs. All rights reserved.

Audit Scope

- **Repository:** https://github.com/neutral-trade/bundle-sc
- **Before Fix Review Commit Hash:** f01546323116b384488369eabbe7f0aa23a12523
- **After Fix Review Commit Hash:** 5fa818084071e94e9617d8dfedbcf52b75b610a5
- **Files/Modules in Scope:**
  - **programs/ntbundle/src/**/*.rs**

© 2026 Adevar Labs. All rights reserved.

2. Assumptions and Considerations

Audit Assumptions

- The manager, keeper and master admin keys are treated as honestly operated. Key compromise is out of scope, except where a finding states that a role can misuse its own permissions.
- The equity figure that the keeper writes through **update_oracle** is an off-chain valuation. The audit did not verify it against the strategy receivers' actual holdings.
- Strategy receivers, including the custodians that hold allocated funds off-chain, were not audited. The behavior of their systems is out of scope.
- Several accepted findings are safe only because the keeper calls instructions at a sensible time. Resolving the oracle deadlock also removed the staleness bound on the pool fee crank, the instant mint path and redemption requests, so the keeper owns that timing.
- Deployment facts, such as live bundle counts, fee rates and account balances, come from chain reads dated 2026-09-21, 2026-09-23 and 2026-10-02. Where a fix review note re-measures a figure, it gives the more recent date. These facts may have changed since.

© 2026 Adevar Labs. All rights reserved.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions convey our team's understanding of both the potential consequences of a vulnerability and how likely it is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easily an attacker can discover and exploit a vulnerability, as well as how economically attractive the exploit is.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risk.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but the financial risk remains manageable.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically infeasible or requires highly specialized conditions. They may require extraordinary effort or entail significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

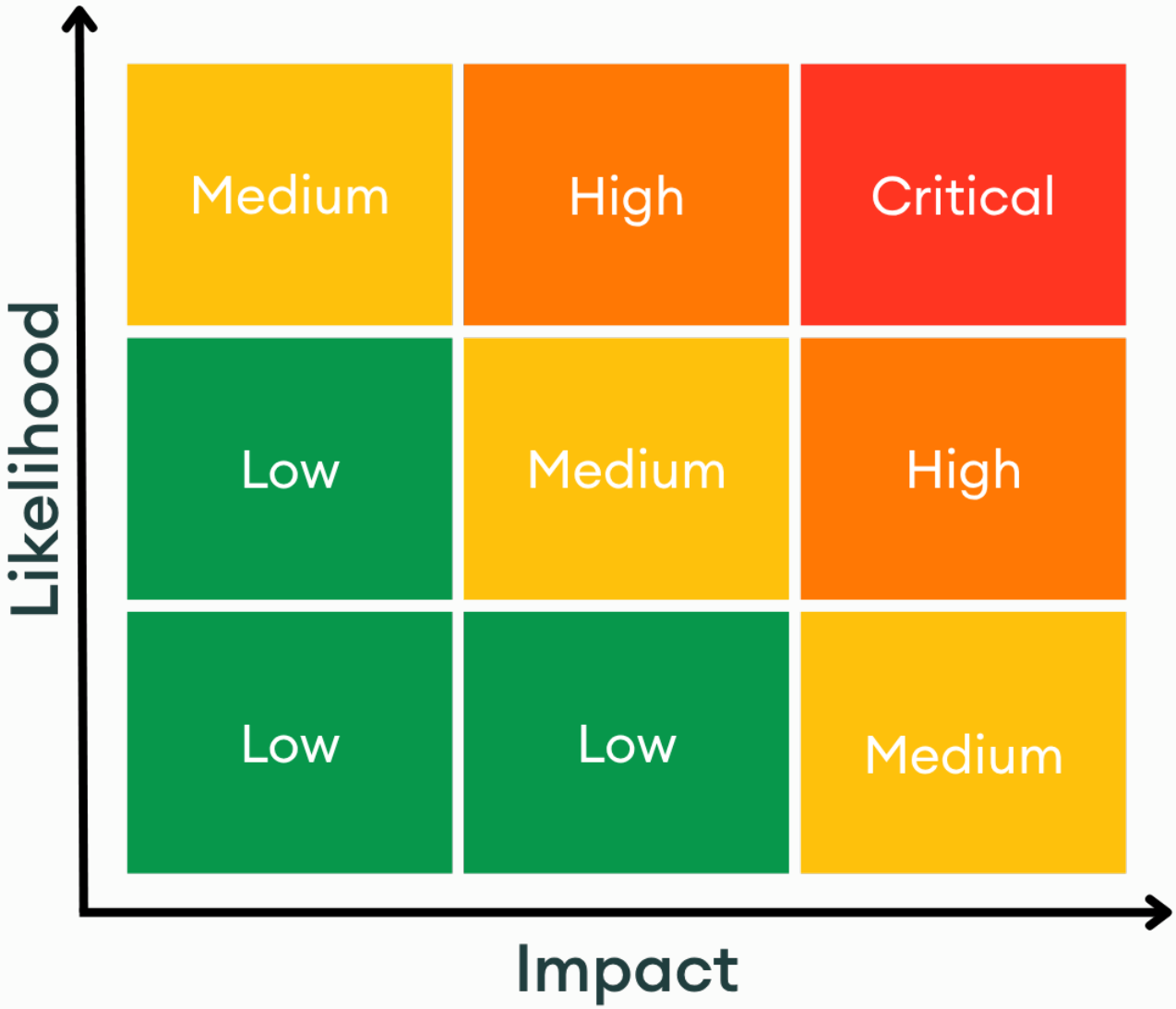
By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact with high likelihood (e.g., a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Medium impact and likelihood, or high impact/likelihood with low likelihood/impact

© 2026 Adevar Labs. All rights reserved.

- **Low**: Low impact and likelihood, or medium impact/likelihood with low likelihood/impact

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

© 2026 Adevar Labs. All rights reserved.

4. Findings

H01: Closing the switch-target account after request_bundle_switch leaves a withdrawal nothing can settle, so the source bundle can never unpause



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/process_switch.rs:L367-L377

RELEVANT SNIPPET

>_ process_switch.rs — RUST

```
365:  pub target_bundle_account: Box<Account<'info, Bundle>>,
366:
367:  #[account(
368:      mut,
369:      seeds = [
370:          USER_BUNDLE_ACCOUNT,
371:          user_bundle_account_owner.key().as_ref(),
372:          target_bundle_account.key().as_ref()
373:      ],
374:      bump,
375:      constraint = target_user_bundle_account.owner == user_bundle_account_owner.key() @
         ErrorCode::SwitchTargetAccountMismatch
376:  )]
377:  pub target_user_bundle_account: Box<Account<'info, UserBundleAccount>>,
378:
379:  #[account(
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/close_user_bundle_account.rs:L19-L34

DESCRIPTION

A bundle switch moves a user's position from source bundle A into the pending queue of target bundle B. **request_bundle_switch** requires the user's **UserBundleAccount** on B to exist, and **process_switch** requires it again at settlement. Nothing keeps it alive in between.

1. **request_bundle_switch** sets **switch_active = true** on the A account and increments A's **pending_withdrawals_counter**. It only reads the B account, which keeps **shares**, **pending_deposit**, **pending_shares** and **estimated_pending_withdrawal_value** at 0.
2. **close_user_bundle_account** checks exactly those four fields on the account it closes, so the user

© 2026 Adevar Labs. All rights reserved.

can close B at once.
3. **process_switch** declares **target_user_bundle_account** as a required **Box<Account<'info, UserBundleAccount>>**. Anchor rejects the closed account with **AccountNotInitialized** before the handler runs, so **fallback_switch_to_wallet** is unreachable.

No other path settles the withdrawal: **process_withdrawal** requires **!switch_active**, and no other instruction clears a switch. **pause_deposits_withdrawals(false)** requires **pending_withdrawals_counter == 0**, so A stays paused and every holder loses deposits, withdrawals and distributions.

Only the user can re-create the B account, since **initialize_bundle_depositor** requires **authority** to sign. On a permissioned B the target manager can, so the user picks one of the 23 non-permissioned USDC bundles. The cost is one deposit into A, 100 USDC on NT-Master.

PROOF OF CONCEPT

1. Alice holds shares in bundle A and an empty **UserBundleAccount** on non-permissioned bundle B.
2. Alice calls **request_bundle_switch(A → B)**. A's **pending_withdrawals_counter** becomes 1.
3. Alice calls **close_user_bundle_account** on B. The account closes.
4. The keeper pauses and nets A, then calls **process_switch**. It reverts with **AccountNotInitialized** on **target_user_bundle_account**.
5. **process_withdrawal** reverts with **SwitchAlreadyActive**, and **pause_deposits_withdrawals(false)** reverts with **PendingTransactionsExist**.
6. Neither the keeper nor B's manager can re-create the account. A stays paused with the counter at 1 until Alice re-creates it.

Full runnable PoC, passing against the deployed mainnet program on a Surfpool fork:

https://github.com/AdevarLabs/neutral-trade-20260911/tree/main/poc/H01-switch-target-close

RECOMMENDATION

Make **target_user_bundle_account** an **Option<Box<Account<'info, UserBundleAccount>>>** and fall back to the wallet when it is absent. The keeper must pass the program id as a placeholder, since **allow-missing-optionals** covers trailing accounts only.

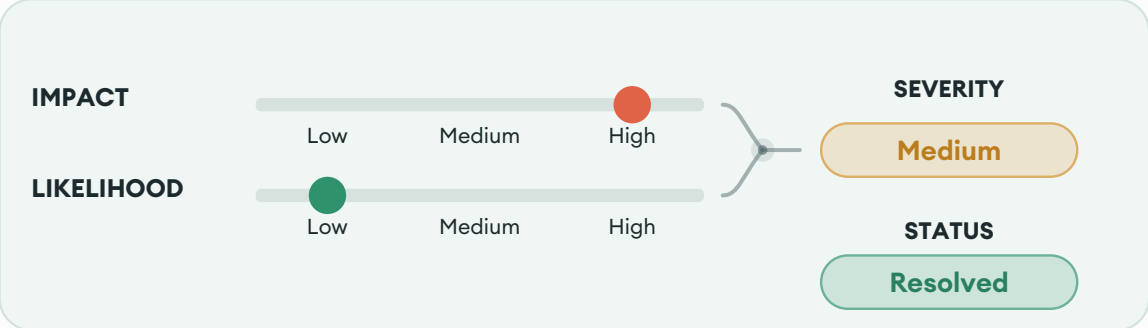
Add **SWITCH_FALLBACK_TARGET_MISSING: u8 = 4**, leave **resolve_switch_withdrawal_fee_bps**'s signature alone and branch at the call site, and charge the full source withdrawal fee in the **None** arm.

FIX REVIEW NOTES

> Fixed. **process_switch** now loads the target account itself and settles to the user's wallet at the source bundle's full withdrawal rate when that account is gone.

© 2026 Adevar Labs. All rights reserved.

M01: Fee rate changes apply to elapsed time and accrued gain, so raising a rate charges holders for a closed window



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/set_fees.rs:L17-L33

RELEVANT SNIPPET

>_ set_fees.rs — RUST

```rust
let bundle_account = &mut ctx.accounts.bundle_account;

require!(deposit_fee <= 5000, ErrorCode::ExcessiveDepositFee);
require!(withdrawal_fee <= 5000, ErrorCode::ExcessiveWithdrawFee);
require!(performance_fee <= 7500, ErrorCode::ExcessivePerformanceFee);
require!(
    management_fee_bps <= 2000,
    ErrorCode::ExcessiveManagementFee
);
require!(
    treasury != Pubkey::new_from_array([0u8; 32]),
    ErrorCode::InvalidReceiver
);

bundle_account.treasury_account = treasury;
bundle_account.deposit_fee = deposit_fee;
bundle_account.withdrawal_fee = withdrawal_fee;
bundle_account.performance_fee = performance_fee;
bundle_account.management_fee_bps = management_fee_bps;

emit!(ChangedCoreParams {
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L1365-L1434

DESCRIPTION

set_fees writes **management_fee_bps** and **performance_fee** and touches nothing else. Both rates are read live at charge time by **resolve_effective_fees**. Neither the elapsed-time counter nor the watermark lives on an account the setter loads, so nothing settles the closing window at the old rate.

Management fee: **charge_bundle_share_holder_fees_at_time** computes **seconds_elapsed** against **holder.last_management_fee_timestamp()** and multiplies by today's rate. Exposure equals the crank gap, so a keeper cranking daily bounds this half to one day.

© 2026 Adevar Labs. All rights reserved.

Performance fee: a raised rate prices all gain above the holder's last committed watermark. At **performance_fee = 0** that watermark never advances (**L01: At a zero performance fee the high-water mark never commits, so a later rate rise prices gain against a stale mark**), so no crank cadence helps: raising the rate then charges every gain earned while the fee was zero, at up to 7500 bps.

Read from chain on 2026-09-21, fourteen of the twenty-nine live bundles carry **performance_fee = 0**, so they accrue gain behind a frozen watermark today. Rates are also demonstrably changed after deployment: five bundles run **management_fee_bps = 1200**, a rate no script in the repository sets (the scripts use **0** or **200**), so **set_fees** is part of normal operation rather than an exceptional act.

The program offers no way to apply a new rate prospectively. A manager who wants to charge only from the change forward cannot.

set_fees is manager-signed. **apply_fees_to_user** and **apply_fees_to_tokenized_depositor** are keeper-signed. None of the three carries a pause gate, so they fit in one transaction. Five deployed bundles run **manager == keeper** (**L04: Bundles where the manager and keeper share one key freeze permanently if that key stops**).

Trigger: any **set_fees** call that raises a rate above what applied during the accrual window.

PROOF OF CONCEPT

Alice deposits 1,000 USDC into a bundle running **PERFORMANCE_FEE = 0**. **process_deposit** sets her **hwm_per_share** to the entry price of 1.00.

1. Over a year the fund grows 50% and the share price reaches 1.50.
2. Every crank calls **calculate_performance_fee_shares** with **performance_fee_bps = 0**, which returns 0.
3. **if performance_fee_shares > 0** is false on every crank, so **commit_performance_fee_charge** never runs and **hwm_per_share** stays at 1.00.
4. **process_deposit**'s blend is gated on the price being below the watermark, so 1.50 never lowers it.
5. The manager calls **set_fees** with **performance_fee = 7500**.
6. The keeper calls **apply_fees_to_user** in the same transaction.

profit_per_share = 1.50 - 1.00 = 0.50. On Alice's 1,000 shares that is 500 USDC of gain, and 375 USDC is charged at once.

Alice loses 375 USDC, 25% of her 1,500 position. The entire charge covers a period in which the on-chain rate was zero.

Boundary: had the rate been 7500 from the start, every crank would have committed the watermark forward and Alice would have paid the same 375 USDC in increments. The totals match, and that is the point. The defect is not an arithmetic overcharge. It is that the charge lands against a closed window under terms that did not apply then, and no path exists to charge from the change forward.

RECOMMENDATION

Stamp the rate change on the bundle and have both accruals respect it.

© 2026 Adevar Labs. All rights reserved.

```rust
// Bundle
pub last_fee_change_timestamp: i64,

// set_fees, before writing the new rates
bundle_account.last_fee_change_timestamp = Clock::get()?.unix_timestamp;
```

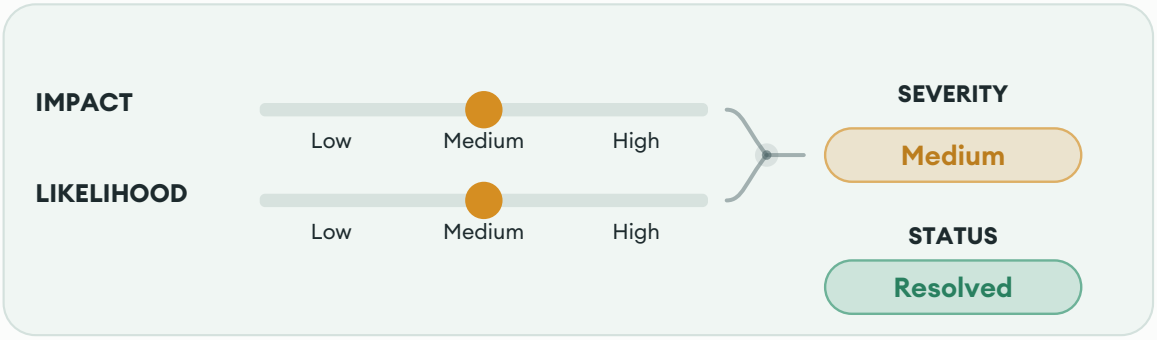
Clamp the management window to **max(last_management_fee_timestamp, last_fee_change_timestamp)**, and advance **hwm_per_share** to the current price when the performance rate changes, even where the charged shares are zero.

FIX REVIEW NOTES

Fixed. Management fees now accrue only from the rate change, and performance fees reset the holder's basis at the next settlement. The team accepts the accrual forfeited for the closing window as intentional, and we agree it is the safe direction.

© 2026 Adevar Labs. All rights reserved.

M02: Oracle refresh requires the pause that the tokenized path forbids, halting tokenized deposits between netting cycles



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/update_oracle.rs:L8-L77

RELEVANT SNIPPET

>_ update_oracle.rs — RUST

```rust
use crate::events::*;

pub fn update_oracle(ctx: Context<SetOracleData>, new_equity: u64) -> Result<()> {
    let oracle_data = &mut ctx.accounts.oracle_data;
    let bundle_data = &mut ctx.accounts.bundle_account;
    let bundle_temp_data = &mut ctx.accounts.bundle_temp_data;

    // ensure deposits and withdrawals are paused
    require!(
        bundle_temp_data.paused_deposits_withdrawals,
        ErrorCode::UnpausedDepositsWithdrawals
    );

    let current_equity = oracle_data.average_external_equity;

    if bundle_data.oracle_update_time_limit != 0 {
        // Check oracle update frequency constraint
        let current_time = Clock::get()?.unix_timestamp;
        let time_since_last_update = current_time
            .checked_sub(oracle_data.last_update_time)
            .ok_or(ErrorCode::MathError)?;

        require!(
            time_since_last_update >= bundle_data.oracle_update_time_limit,
            ErrorCode::OracleUpdateTooFrequent
        );
    }

    let current_equity_buffer_u128 = (current_equity as u128)
        .checked_mul(bundle_data.oracle_buffer as u128)
        .ok_or(ErrorCode::MathError)?
        .checked_div(100)
        .ok_or(ErrorCode::MathError)?;
    let current_equity_buffer =
        u64::try_from(current_equity_buffer_u128).map_err(|_| ErrorCode::MathError)?;

    msg!("Current equity: {}", current_equity);
```

© 2026 Adevar Labs. All rights reserved.

>_ update_oracle.rs RUST

```rust
    msg!(
        "Current equity: {}, Current equity buffer: {}",
        current_equity,
        current_equity_buffer_u128
    );

    // new equity cannot exceed / subceed current equity by the buffer set
    if current_equity != 0 {
        require!(
            new_equity
                <= current_equity
                    .checked_add(current_equity_buffer)
                    .ok_or(ErrorCode::MathError)?,
            ErrorCode::EquityOutOfBufferBound
        );
        require!(
            new_equity
                >= current_equity
                    .checked_sub(current_equity_buffer)
                    .ok_or(ErrorCode::MathError)?,
            ErrorCode::EquityOutOfBufferBound
        );
    }

    let average_external_equity = update_oracle_data(oracle_data, new_equity)?;

    emit!(OracleUpdated {
        average_external_equity: average_external_equity,
        bundle_account_key: bundle_data.key(),
        total_shares: bundle_data.total_shares,
        bundle_underlying_balance: bundle_data.bundle_underlying_balance,
    });

    Ok(())
}

#[derive(Accounts)]
```

DESCRIPTION

update_oracle requires **paused_deposits_withdrawals**. **instant_mint_bundle_tokens**, **request_token_withdrawal** and **apply_fees_to_tokenized_depositor** require the opposite state, then call **check_oracle_data_freshness**. No keeper-signed instruction can refresh the clock while the tokenized path is callable, because **distribute_to_receivers** is pause-bound through **is_distribution_started**. That leaves receiver-signed **perform_refill** as the only unpaused writer of **last_update_time**, and it calls the freshness check first, so once the oracle expires receivers can no longer return the funds **process_withdrawal** pays users from.

oracle_max_age sets the size of the window. **migrations/cli/initBundle.ts** ships **300**, so the tokenized surface is usable for five minutes after each netting cycle and unusable for the remainder unless a receiver happens to refuel. On chain, 22 of the 29 live bundles run **1800**, thirty minutes (read 2026-10-02). If a full netting cycle runs more often than that, the window never closes. No live bundle has a **TokenizedBundleDepositor**, so this is a blocker to clear before tokenization is enabled.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

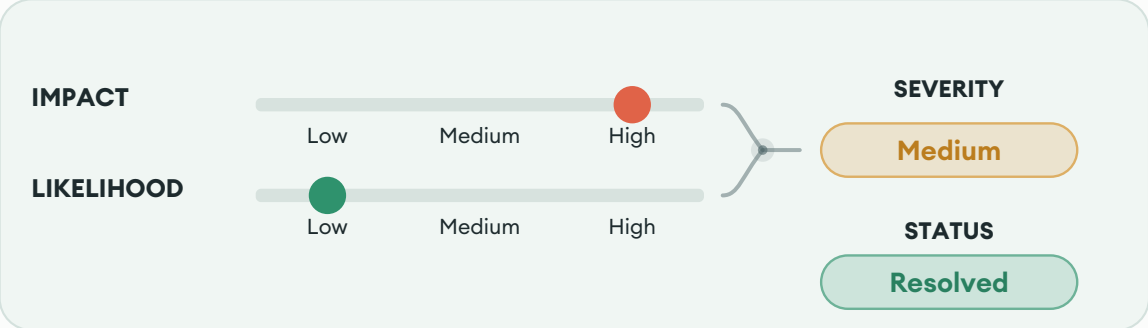
Drop the pause requirement from **update_oracle**, or give the tokenized instructions a staleness bound that a netting cycle refreshes. Restoring the clock today needs a full netting cycle, because **pause_deposits_withdrawals(false)** requires both pending counters at **0**.

FIX REVIEW NOTES

Fixed. Minting and redeeming no longer need a fresh oracle, and the fee crank now matches **apply_fees_to_user** while leaving the bundle share price untouched. The team accepts the remaining price-age risk on the basis that the keeper charges at a sane time, and a non-zero issuance fee should be set before a bundle is tokenized.

© 2026 Adevar Labs. All rights reserved.

M03: Deposit crediting zero shares increments the pending counter and permanently freezes the bundle



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/request_deposit.rs:L85-L119

RELEVANT SNIPPET

>_ request_deposit.rs — RUST

```rust
}

pub(crate) fn apply_pending_deposit_credit(
    user_bundle_account: &mut UserBundleAccount,
    bundle_temp_data: &mut BundleTempData,
    credit: PendingDepositCredit,
) -> Result<()> {
    user_bundle_account.pending_deposit = user_bundle_account
        .pending_deposit
        .checked_add(credit.net_deposit)
        .ok_or(ErrorCode::MathError)?;

    user_bundle_account.total_fee_charged = user_bundle_account
        .total_fee_charged
        .checked_add(credit.deposit_fee)
        .ok_or(ErrorCode::MathError)?;

    bundle_temp_data.cumulative_pending_deposits = bundle_temp_data
        .cumulative_pending_deposits
        .checked_add(credit.net_deposit)
        .ok_or(ErrorCode::MathError)?;

    msg!(
        "Updated - Pending deposit: {}, Cumulative: {}",
        user_bundle_account.pending_deposit,
        bundle_temp_data.cumulative_pending_deposits
    );

    if !credit.topping_up {
        bundle_temp_data.pending_deposits_counter = bundle_temp_data
            .pending_deposits_counter
            .checked_add(1)
            .ok_or(ErrorCode::MathError)?;
    }

    Ok(())
```

© 2026 Adevar Labs. All rights reserved.

>_ request_deposit.rs RUST

```
119: }
120:
121: pub fn request_deposit(ctx: Context<RequestDeposit>, amount: u64) -> Result<()> {
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/refund_deposit.rs:L34-L56

DESCRIPTION

apply_pending_deposit_credit adds **credit.net_deposit** to **pending_deposit**, then increments **pending_deposits_counter** when **topping_up** is false. The increment never asks whether the credit was non-zero, so it counts a request rather than a deposit.

Only two instructions lower that counter, and both refuse an account that received nothing. **process_deposit** requires **pending_deposit != 0** and then **new_subscription_shares != 0**. **refund_deposit** requires **pending_deposit != 0**, then divides by **total_shares_minted**, and **u128::checked_div(0)** returns **None** on the line above the decrement.

Two inputs reach the increment with nothing to decrement later. **Route A:** **calculate_fee_amount_ceil** rounds up, so **amount = 1** leaves **net_deposit = 0** at any non-zero **deposit_fee**. **Route B:** a cycle where **cumulative_pending_deposits * total_shares < total_assets** floors **shares_to_mint** to 0 in **net_pending_transactions**, and Route B runs at the live **deposit_fee = 0**.

With the counter stuck, **pause_deposits_withdrawals(false)** and **start_distribution** both revert on **PendingTransactionsExist**. The bundle stays paused, so no holder can open a redemption. **BundleTempData** uses plain **init** and no instruction resets the counter, so recovery needs a program upgrade.

migrations/cli/initBundle.ts passes **MIN_DEPOSIT_AMOUNT = new BN(0)** on a live call, so a bundle created by the shipped script is open to Route B from birth. **set_min_deposit_amount** writes the value with no validation. A chain read on 2026-09-21 found no production bundle currently exposed: the seven at **min_deposit_amount = 0** are the five deprecated **manager == keeper** bundles plus two test bundles.

PROOF OF CONCEPT

1. A bundle runs **min_deposit_amount = 0**, as **initBundle.ts** creates it.
2. Mallory calls **request_deposit(1)**. **pending_deposits_counter** goes to 1.
3. The keeper pauses and nets. **total_pending_deposits * total_shares < total_assets**, so **shares_to_mint** floors to 0 and no shares are credited.
4. **process_deposit** reverts on **new_subscription_shares != 0**. **refund_deposit** reverts on **checked_div(0)**. The counter stays at 1.
5. **pause_deposits_withdrawals(false)** reverts on **PendingTransactionsExist**. The bundle is paused permanently.

Cost to Mallory: one base unit, about $0.000001 on a 6-decimal asset, plus refundable rent.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Reject a credit that adds nothing, and let the refund path clear a cycle that minted nothing.

RUST

```rust
// prepare_pending_deposit_credit
if net_deposit == 0 { return Err(ErrorCode::DepositAmountZero); }

// refund_deposit, replacing the share-revert arithmetic
if total_shares_minted == 0 {
    (0u128, user_bundle_account.pending_deposit)   // refund in full
} else { /* existing arithmetic */ }
```

Do not guard **shares_to_mint != 0** inside **net_pending_transactions**. That reverts the whole netting, leaves **is_netting_done** false, and blocks **process_deposit**, **refund_deposit** and every other holder's **process_withdrawal** on **NettingNotDone**, which is a wider freeze than the one reported here.

FIX REVIEW NOTES

Fixed. **request_deposit** now rejects a deposit the fee would consume entirely, **refund_deposit** returns the full pending deposit without dividing by a zero share count, and the withdrawal path waits until those refunds finish.

© 2026 Adevar Labs. All rights reserved.

L01: At a zero performance fee the high-water mark never commits, so a later rate rise prices gain against a stale mark



LOCATION

bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L1427-L1433

RELEVANT SNIPPET

>_ bundle.rs — RUST

```rust
msg!("Total fee shares: {}", total_fee_shares);

if performance_fee_shares > 0 {
    holder.commit_performance_fee_charge(
        share_price,
        asset_precision,
        performance_fee_shares,
    )?;
}

if total_fee_shares > 0 {
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/process_deposit.rs:L146

DESCRIPTION

commit_performance_fee_charge runs only when **performance_fee_shares > 0**. The fee is a direct multiple of the rate, so at **performance_fee = 0** the charge is always zero and **hwm_per_share** never moves.

process_deposit re-weights the mark only when **charged_fees.share_price < hwm_value**, relying on that commit having run. A deposit above the stale mark gets neither, so the new shares inherit a mark from before they were bought, and it can sit below the holder's own average entry price.

14 of 29 live bundles run **performance_fee = 0**. On those, a later **set_fees** raise prices gain against the stale mark; **M01: Fee rate changes apply to elapsed time and accrued gain, so raising a rate charges holders for a closed window** walks through the resulting charge.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Advance the mark whenever the price sits above it, not only when a fee is actually charged.

```rust
// BundleFeeShareHolder
fn is_above_watermark(&self, share_price: u128, asset_precision: u128) -> bool;
//   UserBundleAccount:         share_price > self.hwm_per_share
//   TokenizedBundleDepositor:  its value-based profit is positive

// charge_bundle_share_holder_fees_at_time, replacing `if performance_fee_shares > 0`
if holder.is_above_watermark(share_price, asset_precision) {
    holder.commit_performance_fee_charge(...)?;
}
```

FIX REVIEW NOTES

Fixed. A holder's high-water mark now advances to the current price even when the performance fee is zero, so a later rate rise no longer prices gain against a stale mark.

© 2026 Adevar Labs. All rights reserved.

L02: The withdrawal curve is unvalidated on both bundle write paths, while the per-user path guards it



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/set_delays.rs:L30-L32

RELEVANT SNIPPET

>_ set_delays.rs — RUST

```
28:
29: bundle_account.withdrawal_delay = withdrawal_delay;
30: bundle_account.withdrawal_t_min = withdrawal_t_min;
31: bundle_account.withdrawal_t_max = withdrawal_t_max;
32: bundle_account.withdrawal_curve = withdrawal_curve;
33:
34: emit!(DelaysSet {
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/initialize_bundle.rs:L113-L116

DESCRIPTION

set_delays and **initialize_bundle** both write **bundle_account.withdrawal_curve**, an **f32**, with no validation. Each checks only **withdrawal_delay**, **withdrawal_t_max** and the **t_min <= t_max** ordering. The per-user path, **validate_user_withdrawal_timing_override_mask**, requires **custom_withdrawal_curve.is_finite() && >= 0.0**. The same parameter is guarded on the user path and unguarded on both bundle paths. All 29 live bundles were created through **initialize_bundle**, so the create path matters as much as the setter.

calculate_withdrawal_cooldown_time computes **t_min + (t_max - t_min) * ratio^curve** and no caller clamps the result. **ratio <= 1**, so a negative curve makes the factor exceed 1 and the cooldown exceeds **t_max**; a large negative curve overflows **i64** and **request_withdrawal** reverts with **MathError** rather than a named timing error. **borsh** refuses **NaN** on float deserialize, so **NaN** is not storable.

The incremental capability is small. **set_delays** already permits **withdrawal_delay** up to 63,072,000 seconds, so a manager can block every holder's exit for 730 days through a fully validated setting. The curve gap adds a longer cooldown for holders past that delay, and an unnamed error instead of a named one. Recovery is one **set_delays** call with a sane curve, and **withdrawal_available_timestamp** is already stamped on booked withdrawals.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

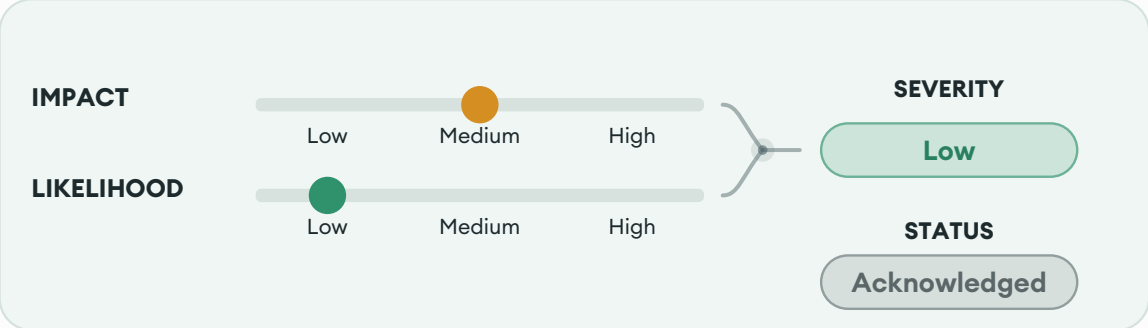
Extract the checks in **validate_user_withdrawal_timing_override_mask** into one helper and call it from **set_delays** and **initialize_bundle**. Reuse **ErrorCode::InvalidWithdrawalCooldownPeriod** rather than adding a variant.

FIX REVIEW NOTES

Acknowledged. Only the manager key can set the withdrawal curve, and we confirmed on chain that all 29 live bundles run a curve of exactly 2.0.

© 2026 Adevar Labs. All rights reserved.

L03: Manager reprices a committed withdrawal by raising the fee rate before settlement



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/request_withdrawal.rs:L22-L32

RELEVANT SNIPPET

>_ request_withdrawal.rs RUST

```
20: let requested_withdrawal = ctx.accounts.request_withdrawal(shares_amount)?;
21:
22: let estimated_withdrawal_fee = calculate_fee_amount_ceil(
23:     requested_withdrawal.estimated_withdrawal_amount,
24:     resolve_effective_fees(
25:         &ctx.accounts.bundle_account,
26:         &ctx.accounts.user_bundle_account,
27:     )
28:     .withdrawal_fee,
29: )?;
30: let estimated_net_amount = requested_withdrawal
31:     .estimated_withdrawal_amount
32:     .checked_sub(estimated_withdrawal_fee)
33:     .ok_or(ErrorCode::MathError)?;
34:
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/process_withdrawal.rs:L48-L54

DESCRIPTION

A user commits an exit at **request_withdrawal**, and the keeper settles it later at **process_withdrawal**. The fee rate is not fixed at commit time.

1. **request_withdrawal** pins **pending_shares** and computes an **estimated_withdrawal_fee** from **resolve_effective_fees(...).withdrawal_fee**. The estimate is emitted in an event, never stored.
2. There is no cancel, which the client confirmed is intended.
3. **process_withdrawal** (Standard mode) reads **resolve_effective_fees(...).withdrawal_fee** again at settle, so the rate in force at settle applies.

Between the two calls the manager raises the rate, and the committed user cannot exit. Two levers do it, both manager-signed:

© 2026 Adevar Labs. All rights reserved.

1. **set_fees** raises the bundle **withdrawal_fee** for everyone.
2. The per-user fee override (**FEE_OVERRIDE_WITHDRAWAL** with **custom_withdrawal_fee_bps**) raises the rate for one named user, leaving all others untouched.

Both setters cap the rate at **5000** bps. So the manager reprices a committed exit up to 50% of its redemption value after the user can no longer back out. On a 100,000 USDC exit at a bundle that showed 0 bps at request, the surprise loss is 50,000 USDC to the treasury.

The per-user override is the sharper vector, since it targets a single exiter without a visible bundle-wide change.

RECOMMENDATION

Pin the rate at request time and settle at exactly that rate. Do not take the lower of the stored and live rates: the field lands in zeroed padding, so **min(0, live)** waives the fee on every withdrawal already booked.

RUST

```rust
pinned_withdrawal_fee_bps: u32,
fee_pinned: bool,        // false on legacy accounts, which use the live rate
```

Pin on all three booking paths, including the switch rate that **prepare_process_switch** recomputes as source minus target.

FIX REVIEW NOTES

Acknowledged. The manager key decides when and how fee rates change, and it can already charge the maximum rate on every future exit. The defect therefore does not expand that key's power.

© 2026 Adevar Labs. All rights reserved.

L04: Bundles where the manager and keeper share one key freeze permanently if that key stops



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/pause_deposits_withdrawals.rs:L99

RELEVANT SNIPPET

>_ pause_deposits_withdrawals.rs — RUST

```
97: #[derive(Accounts)]
98: pub struct PauseDePositsWithdrawals<'info> {
99:     #[account(mut, address = bundle_account.keeper @ ErrorCode::UnauthorizedKeeperActi
    on)]
100:     pub keeper: Signer<'info>,
101:
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/set_keeper.rs:L21-L22

DESCRIPTION

A bundle is unpaused, paused, and settled only by its keeper. **pause_deposits_withdrawals** gates on **address = bundle_account.keeper**, and unpause is the same instruction with **pause = false**. Every **process_*** and **settle_*** is keeper-signed too.

The steps that produce a permanent freeze:

1. The keeper pauses the bundle, or the bundle sits paused mid-cycle.
2. The keeper stops signing.
3. Deposits, withdrawals and switches all require the bundle unpaused, so every user path blocks.
4. Recovery needs a new keeper. **set_keeper** installs one, but it gates on **address = bundle_account.manager**. **change_manager** gates on the manager too.

On the bundles where the manager and keeper are the same key, that one key signs both the pause and its only recovery. If the key stops, **set_keeper** and **change_manager** both reject, since both need the dead key. The master admin gates only **set_bundle_creator** and **change_bundle_master_admin**, so it cannot re-key either role.

The result is a permanent freeze of every depositor's funds on that bundle. Only a program upgrade recovers it, and a revoked upgrade authority makes it unrecoverable.

© 2026 Adevar Labs. All rights reserved.

The five bundles that set both roles to **bu2YJQZCcJzpUQZTine5rBZHwTNVWznGEMRnUHPTMRv**, read on 2026-09-23:

bundle	shares	value
Adrena-Bundle	10,509,774,605	$10.6k
AsterFundingArb-USDT-Bundle	567,918,673	$578
Adrena-Bundle-Forest	200,000,000	$200
SSS-Bundle	0	none
The-Great-Debasement-USDC-Bundle	0	none

Bundle accounts:

```
Adrena-Bundle                5XNsfSeuB3SgknCHrbrv47UgcNNUMoG8CBzUJnwPcSej
AsterFundingArb-USDT-Bundle  976iyYPcj5obNC25A4SRP3quDLbDPeXLjF1cgUamMKBK
Adrena-Bundle-Forest         HELqBHiykXecZqYEHSKFh7mzythQoM8CaXQuR6TESdPB
SSS-Bundle                   AKooyojDyjzpYbMq6zWDXzwCyCRaBTGSB6GSpMxsHj57
The-Great-Debasement-USDC-Bundle FCLuh916dbDEK577Wa11CUcqNZhZtFbH6qU7PNfrvDvj
```

The team confirms these bundles are deprecated and inactive, and the total exposed across all five is about $11.4k. **AsterFundingArb-USDT-Bundle** has read paused and netted for 338 days. That deprecation and that total are why this sits at Low, not the mechanism.

Where **manager != keeper**, the manager re-keys and a fresh keeper unpauses, so that case is a recoverable liveness gap. The program has no manager-direct unpause and no timeout, so even the recoverable case leaves a freeze window.

RECOMMENDATION

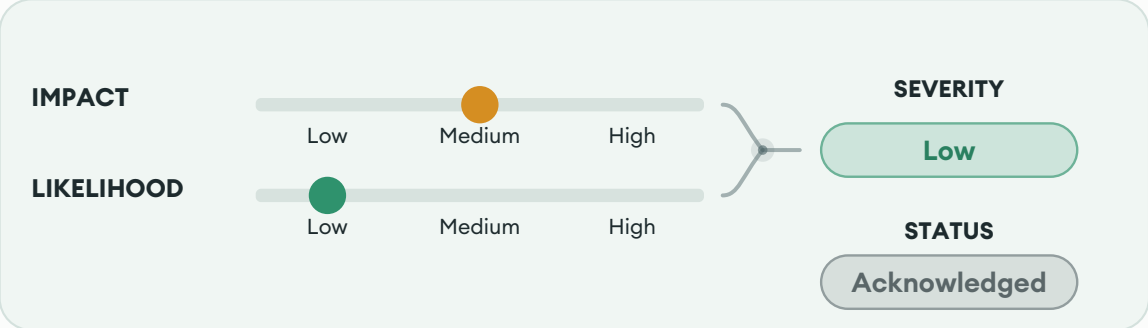
Add a recovery path that does not depend on the frozen key: let the master admin re-key the keeper, or add a manager-direct unpause and an auto-unpause timeout. Forbid setting the keeper equal to the manager.

FIX REVIEW NOTES

Acknowledged. Re-measured on 2026-10-02: five bundles run the manager and keeper on one key, holding about $11,400 between them. Calling **set_keeper** with a separate key removes the risk while the shared key keeps working.

© 2026 Adevar Labs. All rights reserved.

L05: Per-user withdrawal-fee override applies to any receipt token its holder redeems, so exits route around the treasury fee



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/request_token_withdrawal.rs:L129-L138

RELEVANT SNIPPET

>_ request_token_withdrawal.rs — RUST

```
127:
128: let export_value = calculate_assets_from_shares(shares, total_assets, total_shares)?;
129: export_pool_shares_to_user(
130:     &mut accounts.tokenized_bundle_depositor,
131:     &mut accounts.user_bundle_account,
132:     shares,
133:     export_value,
134:     charged_pool_fees.share_price,
135:     current_timestamp,
136: )?;
137:
138: let booked_withdrawal = book_standard_withdrawal(
139:     &mut accounts.user_bundle_account,
140:     &mut accounts.bundle_temp_data,
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/process_withdrawal.rs:L49-L54

DESCRIPTION

request_token_withdrawal moves the redeemed pool shares into the signer's own **UserBundleAccount**, and **process_withdrawal** settles them. Its Standard arm reads the fee from the redeemer's account:

RUST

```
resolve_effective_fees(bundle_account, user_bundle_account).withdrawal_fee
```

resolve_effective_fees returns **custom_withdrawal_fee_bps** when **FEE_OVERRIDE_WITHDRAWAL** is set. Classic shares are not transferable, so an override only ever covers the position it was granted for. Receipt tokens are transferable, so the override now covers any position routed through the holder's account. A holder with a 0 bps override redeems anyone's tokens fee free, and buys them below the bundle rate. The treasury loses that rate on all relayed volume.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Reject the token redemption path for an account carrying the override, mirroring the rule the team already documented on the mint side.

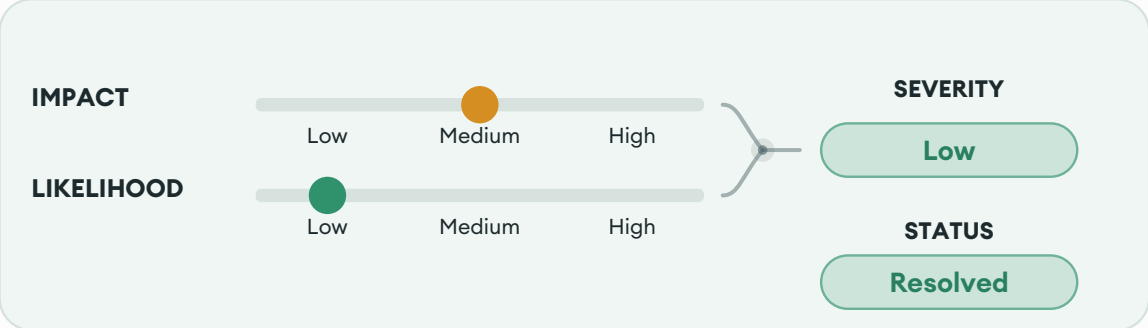
```rust
require!(user_bundle_account.fee_override_flags & FEE_OVERRIDE_WITHDRAWAL == 0, ...);
```

FIX REVIEW NOTES

Acknowledged. The impact is limited to treasury fee revenue, and the path is unreachable until a bundle charging a withdrawal fee is tokenized.

© 2026 Adevar Labs. All rights reserved.

L06: Instant mint never credits the referrer whose book the tokenized exit later debits in full



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/instant_mint_bundle_tokens.rs:L141-L147

RELEVANT SNIPPET

>_ instant_mint_bundle_tokens.rs — RUST

```
139:     .bundle_account
140:     .total_shares
141:     .checked_add(shares_minted)
142:     .ok_or(ErrorCode::MathError)?;
143:
144: accounts.tokenized_bundle_depositor.shares = accounts
145:     .tokenized_bundle_depositor
146:     .shares
147:     .checked_add(shares_minted)
148:     .ok_or(ErrorCode::MathError)?;
149: accounts.tokenized_bundle_depositor.net_deposits = accounts
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/request_token_withdrawal.rs:L295-L318

DESCRIPTION

update_referred_net_deposits has four callers: **process_deposit**, **process_withdrawal**, **refund_deposit** and **close_user_bundle_account**. The tokenized entry is not among them.

instant_mint_bundle_tokens carries no **UserBundleAccount**, so a mint credits no referrer. **export_pool_shares_to_user** then moves the shares into the caller's **UserBundleAccount**, and settlement applies the full **-net_redemption_amount** to the bound referrer. The book falls by principal that entry never added. **set_user_referrer** needs only the caller's signature and an active referrer, so any party can bind to a chosen referrer and repeat this. Once the book goes negative **resolve_bundle_referral_rates** pays a zero split, and nothing recomputes the field.

RECOMMENDATION

Credit **referred_net_deposits** on import in **export_pool_shares_to_user**, mirroring the debit that settlement will later apply, or track the imported portion on **UserBundleAccount** and exclude it from the withdrawal-side delta.

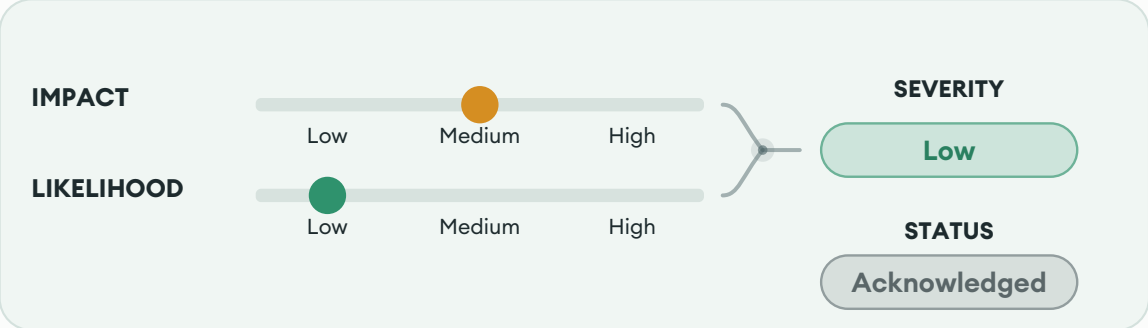
© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Fixed. The redemption request now credits the referrer with **export_value**, so the debit applied at settlement cancels out.

© 2026 Adevar Labs. All rights reserved.

L07: Withdrawal fee transfer to the treasury has no zero-amount guard, so a dead treasury halts every redemption



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/process_withdrawal.rs:L322-L337

RELEVANT SNIPPET

>_ process_withdrawal.rs — RUST

```
320: );
321:
322: let cpi_accounts = TransferChecked {
323:     from: ctx.accounts.bundle_asset_account.to_account_info(),
324:     to: ctx.accounts.treasury_account.to_account_info(),
325:     authority: ctx.accounts.bundle_asset_authority.to_account_info(),
326:     mint: ctx.accounts.asset_address.to_account_info(),
327: };
328: let cpi_ctx = CpiContext::new_with_signer(
329:     ctx.accounts.token_program.to_account_info(),
330:     cpi_accounts,
331:     signer_seeds,
332: );
333: token::transfer_checked(
334:     cpi_ctx,
335:     settled_withdrawal.withdrawal_fee,
336:     settled_withdrawal.asset_decimals,
337: )?;
338:
339: msg!(
```

DESCRIPTION

process_withdrawal ends with two transfers: the net redemption to the user, then the withdrawal fee to **treasury_account**. The second is unconditional, while **request_deposit** guards its equivalent with **if pending_deposit_credit.deposit_fee > 0**.

A zero-amount **transfer_checked** unpacks the destination and rejects a frozen account before it reads the amount, so a frozen, closed or wrong-mint treasury reverts the handler. The rollback discards the **pending_withdrawals_counter** decrement, leaving the state described in **L12: No instruction retires an unpayable withdrawal, so recovery depends on an operator workaround**. One treasury serves 15 bundles, which held 51.7% of TVL on 2026-10-02. **set_fees** repoints it with no pause or counter gate, so recovery is one manager transaction per bundle.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Skip the transfer when the fee rounds to zero. **SetFees** carries only **manager** and **bundle_account**, so it cannot check whether the treasury token account is alive.

```rust
if withdrawal_fee > 0 { token::transfer_checked(...)?; }
```

FIX REVIEW NOTES

Acknowledged, since the treasury is the team's own account and one **set_fees** repoints it.
Re-measured on 2026-10-02: the zero-amount guard we proposed covers only the 18 bundles charging no withdrawal fee, which hold under 5% of equity.

© 2026 Adevar Labs. All rights reserved.

L08: Zero total assets reads as bootstrap, so the next deposit mints 1:1 against shares already outstanding



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/net_pending_transactions.rs:L82-L93

RELEVANT SNIPPET

>_ net_pending_transactions.rs — RUST

```
80:
81: // Calculate shares to mint in single-step to minimize precision loss
82: // Formula: shares_to_mint = (total_pending_deposits * total_shares) / total_assets
83: if total_assets > 0 && total_shares > 0 {
84:     // existing shares and assets
85:     shares_to_mint = (total_pending_deposits as u128)
86:         .checked_mul(total_shares)
87:         .ok_or(ErrorCode::MathError)?
88:         .checked_div(total_assets as u128)
89:         .ok_or(ErrorCode::MathError)?;
90: } else {
91:     // no existing shares, use 1:1 ratio
92:     shares_to_mint = total_pending_deposits as u128;
93: }
94:
95: msg!(
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/instant_mint_bundle_tokens.rs:L286-L288

DESCRIPTION

net_pending_transactions and **calculate_bundle_shares_for_instant_mint** both fall back to a 1:1 mint when either side of the share-price ratio is zero. The first guards its proportional path with **if total_assets > 0 && total_shares > 0**; the second returns **net_amount** whenever **total_assets == 0 || total_shares == 0**.

Two states reach that branch. **total_shares == 0** is a real bootstrap. **total_assets == 0** with shares outstanding is a wiped bundle. Nothing separates them, and the netting comment **// no existing shares, use 1:1 ratio** describes only the first.

A depositor entering the second state mints one share per base unit against the surviving float, so their cash backs every pre-existing share. Their claim is **D * D / (S + D)** for a deposit **D** and outstanding float **S**, both in base units. On a bundle the size of NT-Master, **S = 3,421,099,644,276**, a 1,000 USDC deposit

© 2026 Adevar Labs. All rights reserved.

retains 0.29 USDC, a 99.97 percent loss. **refund_deposit** is keeper-signed and gated on **is_netting_done**, so the depositor cannot exit alone.

Only **update_oracle** can zero **fetch_total_equity**, since the other writers conserve the sum or burn matching shares. Its lower bound is **new_equity >= current_equity - current_equity * oracle_buffer / 100**, which admits zero in one call at **oracle_buffer = 100**. Both **set_oracle_buffer** and **initialize_bundle** permit 100. Only the initializer scripts ship 70, and no script calls **setOracleBuffer**. Live buffers are 5, 7, 10, 15, 20, 30 and 70 across the 29 bundles, so the team sets them out of band. No signer needs to be compromised: at **oracle_buffer = 100** an honest keeper reporting a genuine total loss writes 0.

Trigger: a manager sets **oracle_buffer** to 100, a keeper writes equity to zero, and a deposit is queued. No live bundle carries 100, so this is defense in depth rather than a reachable path on the shipped configuration.

RECOMMENDATION

Reject the deposit at request time, where the rejection costs only the caller. **prepare_pending_deposit_credit** holds no share count today, so the params struct must carry one.

RUST

```
pub total_shares: u128,        // NEW in PendingDepositCreditParams

if params.total_assets == 0 && params.total_shares != 0 {
    return Err(ErrorCode::BundleAssetsDepleted);
}
```

Route this error to the wallet fallback on the switch path, and bound **oracle_buffer** below 100 in both **set_oracle_buffer** and **initialize_bundle**. Do not guard inside **net_pending_transactions**, where a revert leaves **is_netting_done** false and blocks every other holder.

FIX REVIEW NOTES

Acknowledged. Reaching this needs equity at exactly zero with shares outstanding, and no live bundle runs the **oracle_buffer** of 100 that allows it. The refund does return the full deposit, but the reference keeper takes that branch only on a zero-share batch, so a wiped bundle needs a manual call.

© 2026 Adevar Labs. All rights reserved.

L09: Pool watermark nets out the performance fee but not the management fee, so every management charge fakes a loss



LOCATION

bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L802-L825

RELEVANT SNIPPET

>_ bundle.rs — RUST

```rust
}

fn commit_performance_fee_charge(
    &mut self,
    share_price: u128,
    asset_precision: u128,
    charged_performance_fee_shares: u128,
) -> Result<()> {
    let profit = calculate_tokenized_depositor_profit(self, share_price, asset_precision)?;
    let profit = u128::try_from(profit).map_err(|_| ErrorCode::MathError)?;
    let charged_performance_fee_value = charged_performance_fee_shares
        .checked_mul(share_price)
        .ok_or(ErrorCode::MathError)?
        .checked_div(asset_precision)
        .ok_or(ErrorCode::MathError)?;
    // Using the post-cap charged value places all uncollected profit in the
    // new watermark, so a cap cannot turn one profit into repeated fees.
    let newly_taxed_profit = profit
        .checked_sub(charged_performance_fee_value)
        .ok_or(ErrorCode::MathError)?;
    self.cumulative_taxed_profit = self
        .cumulative_taxed_profit
        .checked_add(newly_taxed_profit)
        .ok_or(ErrorCode::MathError)?;
    Ok(())
}

fn last_management_fee_timestamp(&self) -> i64 {
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L1427-L1448

© 2026 Adevar Labs. All rights reserved.

DESCRIPTION

TokenizedBundleDepositor measures profit as **shares * share_price - (net_deposits + cumulative_taxed_profit)**, a value. **UserBundleAccount** uses **hwm_per_share**, a price. Removing shares does not move a price watermark, so only the pool is affected.

commit_performance_fee_charge subtracts the charged performance-fee value from the profit it banks, so that fee cancels exactly. The management fee removes shares in the same charge and nothing subtracts it. The pool's measured profit lands at minus the management-fee value, and the pool must earn that back before the manager collects another performance fee.

Two things hold it there:

1. The trait never passes the management-fee amount into **commit_performance_fee_charge**.
2. That call is gated on **performance_fee_shares > 0**, while the shares leave under **total_fee_shares > 0**. A flat year with a management fee skips the commit entirely.

The divergence per charge equals the management fee's asset value, so the steady state is **performance_bps * management_bps * position** per year. It is zero when either fee is zero and zero on any path that never exceeds its watermark.

Trigger: any fee charge on a **TokenizedBundleDepositor** while **management_fee_bps > 0**. Read from chain on 2026-09-21, five live bundles already run **management_fee_bps = 1200**: CTA-USDC-Bundle, TermMax-Bundle, LighterLP-USDC-Bundle, MoonLP-USDC-Bundle and Great-Debasement-USDC-Bundle. The rate precondition is therefore met today, and the remaining precondition is a **TokenizedBundleDepositor** on one of them. Receipt supply is zero on mainnet and devnet, so this is a launch blocker rather than a live leak.

PROOF OF CONCEPT

A classic holder and the pool start identical: 1,000,000 shares at 1.00, cost basis 1,000,000 USDC, management fee 500 bps, performance fee 2000 bps.

RUST

```rust
#[test]
fn pool_management_fee_manufactures_a_loss_that_shields_the_performance_fee() {
    // year one, price flat at 1.00: the management fee takes 5% from each holder
    charge_bundle_share_holder_fees_at_time(
        &mut pool_bundle, &mut pool, pool_fees_cfg, FLAT, SHARES, T1, 0,
    )
    .unwrap();
    charge_bundle_share_holder_fees_at_time(
        &mut user_bundle, &mut user, user_fees_cfg, FLAT, SHARES, T1, 0,
    )
    .unwrap();

    assert_eq!(pool.shares, 950_000_000_000);
    assert_eq!(user.shares, 950_000_000_000);

    // the pool reports a loss it never took; the price watermark does not move
    assert_eq!(
        calculate_tokenized_depositor_profit(&pool, PREC, PREC).unwrap(),
        -50_000_000_000
    );
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
    assert_eq!(user.hwm_per_share, PREC);

    // price rises to 1.10, no time passes, so only the performance fee runs
    let pool_fee = charge_bundle_share_holder_fees_at_time(
        &mut pool_bundle, &mut pool, pool_fees_cfg, UP, SHARES, T1, 0,
    )
    .unwrap();
    let user_fee = charge_bundle_share_holder_fees_at_time(
        &mut user_bundle, &mut user, user_fees_cfg, UP, SHARES, T1, 0,
    )
    .unwrap();

    let usdc = |shares: u128| shares * 1_100_000 / PREC;
    assert_eq!(usdc(user_fee.performance_fee_shares), 19_000_000_000); // classic: 19,000 USDC
    assert_eq!(usdc(pool_fee.performance_fee_shares),  8_999_999_999); // pool:     9,000 USDC
}
```

The manager collects 10,000 USDC less from the pool, exactly 20% of the phantom 50,000 loss.

Boundary: rerun with **management_fee_bps = 0** and both paths charge 19,000 USDC. That is the setting every existing pool watermark unit test uses, which is why none of them catches this.

RECOMMENDATION

Net the management fee out of the watermark as well, and commit on any charge rather than only on a profitable one.

RUST

```
fn commit_performance_fee_charge(
    &mut self,
    share_price: u128,
    asset_precision: u128,
    charged_performance_fee_shares: u128,
    charged_management_fee_shares: u128,   // new
) -> Result<()>;
```

Move the call under **if total_fee_shares > 0**, and keep the arithmetic in **i128** so an under-water pool does not fail **u128::try_from**.

Add a test that charges both fees together and asserts **cumulative_taxed_profit**. No existing test does.

FIX REVIEW NOTES

Fixed. Management-fee shares now leave the pool with their pro-rata share of the watermark, taken from **net_deposits**, with **cumulative_taxed_profit** untouched and classic holders unchanged. We verified that a management charge no longer shifts the pool's watermark per share while that watermark is positive, and that splitting the charge cannot collect more than a single call does.

© 2026 Adevar Labs. All rights reserved.

L10: Deposit watermark blend is gated on the live share price, so shares issued at the netting price keep the wrong watermark



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/process_deposit.rs:L146-L165

RELEVANT SNIPPET

>_ process_deposit.rs — RUST

```rust
let hwm_value = user_bundle_account.hwm_per_share;

if charged_fees.share_price < hwm_value {
    let hwm_denominator = user_bundle_account
        .shares
        .checked_add(new_subscription_shares)
        .ok_or(ErrorCode::MathError)?;

    let hwm_numerator = (user_pending_deposit as u128)
        .checked_mul(bundle_account.asset_precision as u128)
        .ok_or(ErrorCode::MathError)?
        .checked_add(
            hwm_value
                .checked_mul(user_bundle_account.shares)
                .ok_or(ErrorCode::MathError)?,
        )
        .ok_or(ErrorCode::MathError)?;

    user_bundle_account.hwm_per_share = hwm_numerator
        .checked_div(hwm_denominator)
        .ok_or(ErrorCode::MathError)?;
}

msg!(
```

DESCRIPTION

process_deposit blends a depositor's performance-fee watermark using two values fixed at different moments. **net_pending_transactions** fixes **last_total_shares_minted** and **cumulative_pending_deposits**, whose ratio is the price the shares were issued at. **process_deposit** then reads **total_assets** live from **oracle_data**, once per user, in a separate transaction.

The blend numerator uses **user_pending_deposit**, the real cost basis. The guard **charged_fees.share_price < hwm_value** uses the live price. When the two disagree the guard skips a blend that should run, and the new shares keep the holder's old watermark. An oracle rise then shields

© 2026 Adevar Labs. All rights reserved.

those shares from the performance fee. An oracle fall makes the holder pay a fee on a gain they never made.

update_oracle requires only **paused_deposits_withdrawals**. It carries no **is_netting_done** check, so nothing in the program orders it before netting. The keeper runbook does place it before netting, which is why the two prices normally agree. That ordering is convention, not an enforced invariant, and the deposit loop spans hundreds of transactions.

Trigger: any **update_oracle** call after **net_pending_transactions** and before a depositor's **process_deposit**.

PROOF OF CONCEPT

Alice holds 1,000 shares at **hwm_per_share = 1.00**. The performance fee is 20%.

1. **net_pending_transactions** runs at a share price of 0.80 and issues her 1,250 shares for 1,000 USDC.
2. The keeper calls **update_oracle**. The share price rises to 1.10.
3. **charge_user_fees** takes 20 USDC on her 1,000 old shares and sets **hwm_per_share = 1.10**. She drops to 981.82 shares.
4. The guard tests **1.10 < 1.10**. False, so the blend is skipped.

The correct blend is **(1,000 USDC + 1.10 * 981.82) / 2,231.82 = 0.9320**. Her 1,250 shares were issued at 0.80 and carry 1.10, so their 375 USDC of gain never pays a fee. The manager loses 75 USDC.

Boundary: drop step 2 and the price stays 0.80. The guard tests **0.80 < 1.00**, the blend runs, the watermark lands at 0.8889, and the 75 USDC is collected.

RECOMMENDATION

Gate the blend on the price the shares were issued at, not the live price.

```rust
let paid = (user_pending_deposit as u128)
    .checked_mul(bundle_account.asset_precision as u128)
    .ok_or(ErrorCode::MathError)?;
let protected = hwm_value
    .checked_mul(new_subscription_shares)
    .ok_or(ErrorCode::MathError)?;

if paid < protected {
```

new_subscription_shares is already proven non-zero above, and the cross-multiplication avoids a second division.

FIX REVIEW NOTES

Acknowledged. The proposed guard closes the leak when the oracle moves up but would overcharge the holder when it moves down, so neither comparison is right on its own. The keeper's runbook already orders the oracle update before netting, which keeps the two prices equal in practice.

© 2026 Adevar Labs. All rights reserved.

L11: Deactivating a referrer freezes their referred total instead of letting withdrawals reduce it



LOCATION

bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L489-L493

RELEVANT SNIPPET

>_ bundle.rs — RUST

```
487:  validate_referrer_account(bundle_key, user_bundle_account, referrer_account)?;
488:
489:  if referrer_account.active {
490:      referrer_account.referred_net_deposits = referrer_account
491:          .referred_net_deposits
492:          .checked_add(delta)
493:          .ok_or(ErrorCode::MathError)?;
494:  }
495:
```

DESCRIPTION

update_referred_net_deposits applies a signed delta, and **referred_net_deposits** selects the referrer's fee tier in **resolve_bundle_referral_rates**. Four callers supply that delta: **process_deposit** sends the deposit, **process_withdrawal** sends **checked_neg()** of the redemption, and **refund_deposit** and **close_user_bundle_account** send negatives.

The **if referrer_account.active** gate wraps the accumulator, so it blocks the negatives as well as the positives. While a referrer is deactivated their referred total freezes rather than falling, and their referred users can withdraw everything without moving it.

Nothing recomputes the total, and no setter writes **referred_net_deposits** directly. Reactivating the referrer restores the tier they held before their users left, permanently. On a bundle charging a 2% management fee with a top tier at a 50% split, a referrer holding a stale $10M total keeps a rate they no longer earn.

The manager triggers it through **set_referrer_active**, so the referrer cannot reach it alone. **set_referrer_rate_override** pins a rate directly and bypasses the tier, which is the available remedy.

Confirmed as unintended by the client on 2026-09-18: the referred total should keep counting withdrawals down while a referrer is deactivated, and only the increase should be ignored.

Trigger: the manager deactivates a referrer, referred users withdraw, the manager reactivates.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Gate only the positive direction, so withdrawals keep reducing the total.

```rust
if referrer_account.active || delta < 0 {
    referrer_account.referred_net_deposits = referrer_account
        .referred_net_deposits.checked_add(delta).ok_or(ErrorCode::MathError)?;
}
```

The existing unit test covers the active case only, as its name **referred_net_deposits_tracks_signed_deltas_for_active_referrers** states. Add a case that deactivates a referrer, applies a negative delta, and asserts the total falls.

FIX REVIEW NOTES

Acknowledged: the path is manager-triggered and manager-paid. The active flag gates both legs of the referral book write, so reactivating a referrer mid-redemption is the same defect in reverse. Hold the toggle in both directions while a redemption is pending.

© 2026 Adevar Labs. All rights reserved.

L12: No instruction retires an unpayable withdrawal, so recovery depends on an operator workaround



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/process_withdrawal.rs:L79-L82

RELEVANT SNIPPET

>_ process_withdrawal.rs — RUST

```
77:  );
78:
79:  bundle_temp_data.pending_withdrawals_counter = bundle_temp_data
80:      .pending_withdrawals_counter
81:      .checked_sub(1)
82:      .ok_or(ErrorCode::MathError)?;
83:
84:  let current_timestamp = Clock::get()?.unix_timestamp;
```

DESCRIPTION

request_withdrawal and **request_token_withdrawal** raise **pending_withdrawals_counter**. **settle_pending_withdrawal** lowers it in memory before it knows whether the payout will happen, so a reverting settlement rolls the decrement back and **pause_deposits_withdrawals** refuses to unpause. A withdrawal that is merely locked does not pin the bundle: it decrements and carries over in **pending_locked_withdrawals_counter**. The holder is stranded instead, blocked from re-requesting by **PendingWithdrawalExists** and from closing by **UserBundleAccountNotEmpty**.

No instruction retires a withdrawal that cannot be paid. The keeper retries the same settlement forever.

The team can clear every failing condition, so the freeze is always recoverable:

1. Stale oracle: update the oracle while paused.
2. **InsufficientFunds**: have the receiver refill.
3. Missing referrer account: pass it, or disable referrals.
4. Treasury rejects the fee: repoint the treasury.
5. Frozen payout account: the payout account (**user_token_account**) is constrained only by owner and mint, not derived as the ATA. The keeper creates a fresh token account owned by the user and pays into it. SPL Token allows creating an account for any owner without that owner's signature, so a frozen ATA does not trap the withdrawal.

The gap is that recovery needs a workaround rather than a direct instruction, so a cancel path would be cleaner.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Add a keeper-callable instruction that retires an unpayable withdrawal and returns the user's shares. The deleted **force_withdraw_position** also paid out, so the replacement must not transfer.

RUST

```
bundle_temp_data.pending_withdrawals_counter =
    bundle_temp_data.pending_withdrawals_counter.checked_sub(1).ok_or(ErrorCode::MathError)?;
user_bundle_account.pending_shares = 0;
user_bundle_account.estimated_pending_withdrawal_value = 0;
user_bundle_account.withdrawal_available_timestamp = 0;
```

FIX REVIEW NOTES

Acknowledged as operator-recoverable. The keeper now creates the payout token account before refunds and withdrawals. Apply the same step before the switch and referrer payouts, which accept any token account the user owns on that mint.

© 2026 Adevar Labs. All rights reserved.

L13: Pool loss carry does not scale on exit, so remaining token holders inherit an untaxed fee shield



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/request_token_withdrawal.rs:L311-L315

RELEVANT SNIPPET

>_ request_token_withdrawal.rs — RUST

```
309:     .ok_or(ErrorCode::MathError)?;
310:
311: let export_value = i128::from(export_value);
312: tokenized_bundle_depositor.net_deposits = tokenized_bundle_depositor
313:     .net_deposits
314:     .checked_sub(export_value)
315:     .ok_or(ErrorCode::MathError)?;
316: user_bundle_account.net_deposits = user_bundle_account
317:     .net_deposits
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L784

DESCRIPTION

TokenizedBundleDepositor prices its performance fee against a value watermark, **net_deposits + cumulative_taxed_profit**, read by **calculate_tokenized_depositor_profit**.

1. **instant_mint_bundle_tokens** adds the deposit to **net_deposits**.
2. On a gain, **commit_performance_fee_charge** adds **profit** minus the charged fee to **cumulative_taxed_profit**, so the watermark lands exactly on the post-fee pool value.
3. On a loss, **calculate_performance_fee_shares** returns **0** and writes nothing, so the shortfall persists as an untaxed loss carry.
4. **export_pool_shares_to_user** subtracts the redeemer's **shares** from **shares** and their market value from **net_deposits**.

Step 4 removes the redeemer's value and none of their share of the carry. The carry is therefore invariant under every export. Holders who stay inherit a fee shield they did not earn, and the manager forgoes **performance_fee_bps** times the abandoned amount.

UserBundleAccount.hwm_per_share is a per-share price, so a partial exit scales it with no extra work. The pool stores a total, which the export never scales.

© 2026 Adevar Labs. All rights reserved.

A full wind-down does not clear it either: **reset_tokenized_depositor_basis_if_truly_empty** requires **token_supply == 0** and **shares == 0** together, so one un-redeemed token base unit blocks it permanently and hands the whole shield to the next minter.

Trigger: any redemption while the pool sits below its watermark.

Reachability: the harm is manager fee revenue only, no user funds, and the tokenized path is pre-launch (zero depositors).

PROOF OF CONCEPT

Alice and Bob each deposit 1,000 USDC. The fund drops 20%, so the pool carries 400 of untaxed loss, 200 belonging to each. Alice redeems her half and receives 800. The fund then recovers to 1.20.

Add to **fee_override_tests** in **bundle.rs**. It calls the real **calculate_performance_fee_shares** on both **TokenizedBundleDepositor** and **UserBundleAccount**, with the same position on each.

RUST

```rust
#[test]
fn abandoned_pool_carry_shields_the_remaining_holder_from_the_fee() {
    const PREC: u128 = 1_000_000; // USDC, 6 decimals
    const LOW: u128 = 800_000; // 0.80, after a 20% drawdown
    const HIGH: u128 = 1_200_000; // 1.20, after the recovery
    const PFEE: u32 = 2_000; // 20%

    let mut pool = sample_tokenized_depositor();
    pool.shares = 2_000_000_000; // 2,000 shares
    pool.net_deposits = 2_000_000_000; // watermark = 2,000 USDC

    // the fund is under water, so no fee is charged and the carry is recorded
    assert_eq!(
        calculate_tokenized_depositor_profit(&pool, LOW, PREC).unwrap(),
        -400_000_000,
        "pool is 400 under water: 200 Alice, 200 Bob"
    );

    // exactly what export_pool_shares_to_user does for Alice's half:
    pool.shares -= 1_000_000_000; //   shares       -= shares
    pool.net_deposits -= 800_000_000; //   net_deposits -= export_value

    // her 200 did not leave with her
    assert_eq!(
        calculate_tokenized_depositor_profit(&pool, LOW, PREC).unwrap(),
        -400_000_000
    );

    // Bob alone on the classic path, same 1,000 shares bought at 1.00
    let mut bob = sample_user();
    bob.shares = pool.shares;
    bob.hwm_per_share = PREC;

    let pool_fee = pool.calculate_performance_fee_shares(HIGH, PREC, PFEE).unwrap();
    let classic_fee = bob.calculate_performance_fee_shares(HIGH, PREC, PFEE).unwrap();
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
    let usdc = |shares: u128| shares * HIGH / PREC;
    assert_eq!(usdc(classic_fee), 40_000_000, "classic path: 40 USDC owed");
    assert_eq!(pool_fee, 0, "pool path: the manager collects nothing");
    assert_eq!(usdc(classic_fee) - usdc(pool_fee), 40_000_000);
}
```

Bob holds 1,000 shares bought at 1.00 and the price is 1.20 in both runs. The classic path charges 40 USDC. The pool charges 0, because Alice's abandoned 200 lifted the watermark from 1,000 to 1,200. The gap is **performance_fee_bps** times the abandoned carry.

RECOMMENDATION

Keep the dollar watermark and disclose its incidence limit. Document three properties for token holders and carry them in the risk register:

1. The manager under-collects performance fee by **performance_fee_bps** times any carry a leaver abandons.
2. A holder who redeems under water is re-based at the exit price, and pays performance fee on recovery between request and settlement.
3. The carry persists until contributed dollars recover or the pool truly empties.

Track the abandoned carry as a monitored metric.

FIX REVIEW NOTES

Acknowledged as a property of the dollar watermark, with the disclosure recommended above. Extend that disclosure to receipt-token transfers, since only a per-share fee index closes that gap.

© 2026 Adevar Labs. All rights reserved.

L14: apply_fees_to_user charges fees against an oracle it never checks for staleness



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/apply_fees_to_user.rs:L8-L71

RELEVANT SNIPPET

>_ apply_fees_to_user.rs — RUST

```rust
use crate::events::*;

pub fn apply_fees_to_user<'info>(
    ctx: Context<'_, '_, 'info, 'info, ApplyFeesToUser<'info>>,
) -> Result<()> {
    let bundle_account = &mut ctx.accounts.bundle_account;
    let user_bundle_account = &mut ctx.accounts.user_bundle_account;
    let oracle_data = &ctx.accounts.oracle_data;
    let bundle_key = bundle_account.key();

    let total_assets = fetch_total_equity(&oracle_data, &bundle_account)?;
    let total_shares = bundle_account.total_shares;

    let mut referrer_account = load_referrer_account_from_remaining(
        bundle_key,
        bundle_account,
        user_bundle_account,
        ctx.remaining_accounts,
    )?;

    let charged_fees = charge_user_fees(
        bundle_key,
        bundle_account,
        user_bundle_account,
        referrer_account.as_deref_mut(),
        total_assets,
        total_shares,
    )?;

    if let Some(referrer_account) = referrer_account.as_ref() {
        referrer_account.exit(ctx.program_id)?;
    }

    emit!(ChargedFeesToUser {
        total_fee_shares: charged_fees.total_fee_shares,
        total_fee_value: charged_fees.total_fee_value,
        management_fee_shares: charged_fees.management_fee_shares,
```

© 2026 Adevar Labs. All rights reserved.

>_ apply_fees_to_user.rs RUST

```rust
        performance_fee_shares: charged_fees.performance_fee_shares,
        share_price: charged_fees.share_price,
        bundle_account_key: bundle_key,
        user: charged_fees.user,
        user_shares_after: charged_fees.user_shares_after,
        manager_pfee_shares_after: charged_fees.manager_pfee_shares_after,
        manager_mfee_shares_after: charged_fees.manager_mfee_shares_after,
        referral_pfee_bps: charged_fees.referral_pfee_bps,
        referral_mfee_bps: charged_fees.referral_mfee_bps,
        referral_tier_index: charged_fees.referral_tier_index,
    });

    if charged_fees.referrer_pfee_shares != 0 || charged_fees.referrer_mfee_shares != 0 {
        emit!(ReferralFeesAccrued {
            user: user_bundle_account.owner,
            referrer: user_bundle_account.referrer,
            bundle_account_key: bundle_key,
            referrer_pfee_shares: charged_fees.referrer_pfee_shares,
            referrer_mfee_shares: charged_fees.referrer_mfee_shares,
            accrued_pfee_shares_after: charged_fees.accrued_pfee_shares_after,
            accrued_mfee_shares_after: charged_fees.accrued_mfee_shares_after,
            referral_pfee_bps: charged_fees.referral_pfee_bps,
            referral_mfee_bps: charged_fees.referral_mfee_bps,
            referral_tier_index: charged_fees.referral_tier_index,
        });
    }

    Ok(())
}

#[derive(Accounts)]
```

DESCRIPTION

apply_fees_to_user contains no **require!**, and its account context carries only the **address = bundle_account.keeper** constraint. It passes **fetch_total_equity** into **charge_user_fees** with no freshness check, no pause requirement and no **is_netting_done** gate, while the sibling **apply_fees_to_tokenized_depositor** carries all three. **charge_bundle_share_holder_fees_at_time** moves **total_fee_shares** from the holder to **bundle.manager_pfee_shares** without touching **bundle.total_shares**, so the holder's claim shrinks the moment the crank runs against a stale price, and **commit_performance_fee_charge** writes that price into **hwm_per_share** permanently. **process_deposit** shares the omission, while **settle_pending_withdrawal** in the same paused phase does call **check_oracle_data_freshness**.

RECOMMENDATION

Add the freshness check to both paths, matching **apply_fees_to_tokenized_depositor**.

RUST

```rust
check_oracle_data_freshness(&oracle_data, &bundle_account)?;
```

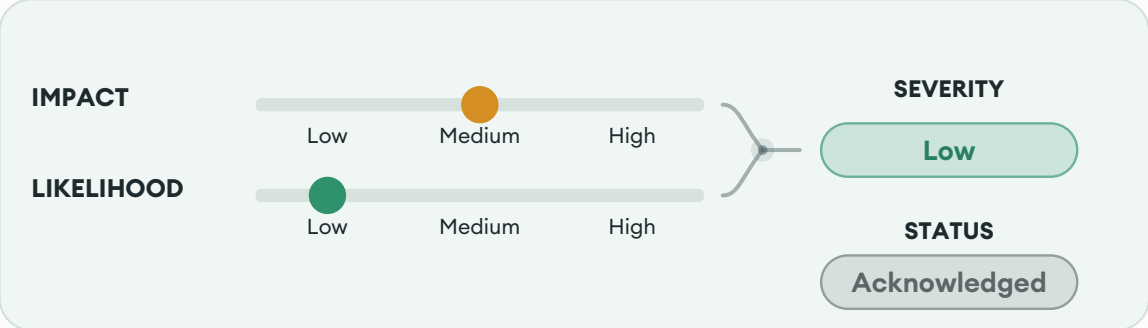
© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Acknowledged. The keeper is the only caller and owns the timing, so we narrow our recommendation to **apply_fees_to_user** alone and withdraw the **process_deposit** half. After the M02 fix, **apply_fees_to_tokenized_depositor** no longer checks freshness either, so both fee cranks rely on the keeper's timing.

© 2026 Adevar Labs. All rights reserved.

L15: Receiver resets the oracle freshness clock and removes the equity buffer through perform_refill



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/perform_refill.rs:L9-L51

RELEVANT SNIPPET

>_ perform_refill.rs — RUST

```rust
use crate::events::*;

pub fn perform_refill(ctx: Context<PerformRefill>, refill_amount: u64) -> Result<()> {
    let bundle_account = &mut ctx.accounts.bundle_account;
    let strategy_account = &ctx.accounts.strategy_account;
    let oracle_data = &mut ctx.accounts.oracle_data;

    // ensure the receiver is allowed
    require!(strategy_account.allowed, ErrorCode::MustBeInWhitelist);

    check_oracle_data_freshness(&oracle_data, bundle_account)?;

    // handle the transfer
    let net_amount_transfer_accounts = TransferChecked {
        from: ctx.accounts.receiver_ata.to_account_info(),
        to: ctx.accounts.bundle_asset_account.to_account_info(),
        authority: ctx.accounts.receiver_address.to_account_info(),
        mint: ctx.accounts.asset_address.to_account_info(),
    };
    let net_amount_context = CpiContext::new(
        ctx.accounts.token_program.to_account_info(),
        net_amount_transfer_accounts,
    );
    anchor_spl::token::transfer_checked(
        net_amount_context,
        refill_amount,
        bundle_account.asset_decimals,
    )?;

    refill(bundle_account, refill_amount)?;

    let new_equity = (oracle_data.average_external_equity)
        .checked_sub(refill_amount)
        .ok_or(ErrorCode::MathError)?;
    let new_equity = update_oracle_data(oracle_data, new_equity)?;
    msg!("New equity: {}", new_equity);

```

© 2026 Adevar Labs. All rights reserved.

>_ perform_refill.rs RUST

```
44:     emit!(Refilled {
45:         from: ctx.accounts.receiver_address.key(),
46:         refill_value: refill_amount,
47:         timestamp: Clock::get()?.unix_timestamp
48:     });
49:
50:     Ok(())
51: }
52:
53: #[derive(Accounts)]
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L251-L262
- bundle-sc/f015463/programs/ntbundle/src/instructions/update_oracle.rs:L48-L64

DESCRIPTION

perform_refill gates only on **strategy_account.allowed** and **check_oracle_data_freshness**, with no keeper signature, no pause requirement and no rate limit. It ends in **update_oracle_data**, which stamps **last_update_time** unconditionally, so a whitelisted receiver holds the staleness check open indefinitely for one base unit while supplying no new NAV. Repaying the full outstanding amount drives **average_external_equity** to **0**, and **update_oracle** skips both buffer bounds inside **if current_equity != 0**, leaving the next keeper write unbounded. Total equity is conserved in both cases, so the receiver gains nothing directly and the loss depends on the keeper error or downtime these two rails exist to contain.

The keeper's own **update_oracle** resets the clock the same way. A write repeating the current value satisfies both buffer bounds, since **new_equity == current_equity** sits inside **current_equity +/- buffer** at any buffer, and **update_oracle_data** then stamps **last_update_time = now**. **OracleData.last_update_time** therefore records the last write rather than the age of the valuation, so a valuation of any age reads as current. Read on 2026-09-23, PW-USDC-Bundle's last oracle write was 15.2 days old and Ethereal-USDE-Bundle's was 50.1 days old, both against an **oracle_max_age** of 1800 seconds.

RECOMMENDATION

Bound **new_equity** against **bundle_underlying_balance** instead of its own previous value, so **current_equity == 0** is not a special case, and do not stamp **last_update_time** from a repayment that carries no NAV information.

Record the valuation's own as-of time in a separate **nav_as_of_ts** field, passed in with each new valuation, and check freshness against it, leaving **last_update_time** to serve the update rate limit. Adding the field must shrink **OracleData.padding** from 64 to 56 bytes, or all 29 live **OracleData** accounts fail to deserialize.

© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Acknowledged. Only whitelisted custodians can call it, total equity is conserved, and the worst case needs a compromised keeper. Keep the current stamping until **nav_as_of_ts** exists, since this is the only writer of that clock that runs while the bundle is unpaused. A separate **nav_as_of_ts** leaves **last_update_time** serving the oracle-update rate limit, so adding it does not stall oracle updates. All 64 padding bytes read zero on the 29 live oracles, so the new field must be treated as unset at zero or backfilled first.

© 2026 Adevar Labs. All rights reserved.

L16: Switch falling back to the user wallet pays the discounted switch fee instead of the source withdrawal fee



LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/process_switch.rs:L139-L168

RELEVANT SNIPPET

>_ process_switch.rs — RUST

```rust
}

fn fallback_switch_to_wallet(
    accounts: &ProcessSwitch,
    signer_seeds: &[&[&[u8]]],
    prepared: &SettledWithdrawal,
    reason: u8,
) -> Result<()> {
    transfer_net_withdrawal_to_user(
        accounts.token_program.to_account_info(),
        accounts.source_bundle_asset_account.to_account_info(),
        accounts.user_token_account.to_account_info(),
        accounts.source_bundle_asset_authority.to_account_info(),
        accounts.asset_address.to_account_info(),
        signer_seeds,
        prepared,
    )?;
    emit!(BundleSwitchFallbackToWallet {
        user: accounts.user_bundle_account_owner.key(),
        source_bundle: prepared.source_bundle,
        target_bundle: prepared.switch_target_bundle,
        shares_redeemed: prepared.shares_to_withdraw,
        gross_redemption: prepared.redemption_amount,
        net_to_wallet: prepared.net_redemption_amount,
        withdrawal_fee_a: prepared.withdrawal_fee,
        reason,
        source_referred_net_deposits_after: prepared.referred_net_deposits_after,
        timestamp: prepared.current_timestamp,
    });
    emit_redeemed(prepared);
    Ok(())
}

fn switch_fallback_for_pending_deposit_credit_error(err: ErrorCode) -> Option<u8> {
```

© 2026 Adevar Labs. All rights reserved.

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L994-L1003

DESCRIPTION

process_switch moves a position from one bundle to another. **resolve_switch_withdrawal_fee_bps** quotes the fee as the source bundle's **withdrawal_fee** minus the target's, saturating at **0**, and **prepare_process_switch** bakes that rate into **settle_pending_withdrawal** before the target deposit is attempted.

The subtraction is a prepaid future withdrawal fee. It is correct only if the user stays on the platform and pays the target's fee on a later exit.

When the target rejects the credit with **PausedDepositsWithdrawals**, **MinDepositAmountNotMet**, **DepositAmountZero** or **MaxDepositAmountExceeded**, **fallback_switch_to_wallet** sends **net_redemption_amount** to the user's own token account and never re-prices. The user leaves the platform having paid the discount, and the platform never collects the target's fee either. The discount reaches 100% whenever the target's **withdrawal_fee** is at least the source's. **settle_pending_withdrawal** clears **switch_active** before the branch is chosen, so the user repeats it with no limit.

The trigger is a switch aimed at any sibling bundle on the same mint that is paused or full when the keeper processes it. **set_max_deposit_amount** carries no validation, so a bundle parked at **0** or sitting at its cap rejects every deposit for as long as it stays there.

Read from chain on 2026-10-02, eleven live bundles charge a non-zero **withdrawal_fee** of 20 to 50 bps, and together they hold 95.8% of equity, so the bypass is reachable on the deployed configuration. The shipped scripts do not reflect this: **migrations/cli/initBundle.ts** and **migrations/cli/deployBundleAdrena.ts** pass **WITHDRAWAL_FEE = 0**, **migrations/leveragedBundle.ts** initializes its bundle at **WITHDRAWAL_FEE = 100**, and **migrations/cli/setFeesAdrena.ts** stages 30 with its **setFees** call commented out.

PROOF OF CONCEPT

Bundles A and B on USDC, both at **withdrawal_fee = 30** bps, the rate nine live bundles charge. Alice holds 1,000,000 USDC of A. B sits at its **max_deposit_amount**.

1. Alice calls **initialize_bundle_depositor** on B. The only gate is **require!(!bundle_account.permissionned)**, and **initBundle.ts** ships **permissionned = false**.
2. Alice calls **request_bundle_switch** from A to B for her whole position.
3. **resolve_switch_withdrawal_fee_bps** returns **30.saturating_sub(30) = 0**, so **withdrawal_fee = 0** and **net_redemption_amount = 1_000_000** USDC.
4. B rejects with **MaxDepositAmountExceeded**. **fallback_switch_to_wallet** sends 1,000,000 USDC to Alice's wallet.
5. **transfer_withdrawal_fee_to_treasury** transfers **0**.

Result: Alice exits A to her own wallet for free. A plain **request_withdrawal** would have charged 3,000 USDC.

Same run with B at 10 bps: the quote is 20 bps, so Alice pays 2,000 USDC instead of 3,000. Any non-zero fee on B leaves at least a partial bypass.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Charge the source bundle's standard rate on the fallback branch, since the user is exiting rather than switching.

RUST

```
let standard_bps = resolve_effective_fees(source_bundle, source_user).withdrawal_fee;
let top_up = calculate_fee_amount_ceil(prepared.redemption_amount, standard_bps)?
    .saturating_sub(prepared.withdrawal_fee);
```

settle_pending_withdrawal already treated **net_redemption_amount** as final, so the top-up comes out of it. Reverting the switch instead of falling back is the smaller change.

FIX REVIEW NOTES

Fixed. The wallet fallback now charges the source bundle's full withdrawal rate, including any per-user override, and takes the difference out of the payout.

© 2026 Adevar Labs. All rights reserved.

5. Enhancement Opportunities

E01: A strategy receiver cannot be replaced in place, and disabling one also blocks the refill path

LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/perform_refill.rs:L15

RELEVANT SNIPPET

>_ perform_refill.rs — RUST

```
13:
14: // ensure the receiver is allowed
15: require!(strategy_account.allowed, ErrorCode::MustBeInWhitelist);
16:
17: check_oracle_data_freshness(&oracle_data, bundle_account)?;
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/remove_strategy.rs:L14

DESCRIPTION

Strategy is a PDA seeded on **receiver_address**, so no instruction can rewrite that field in place. Replacing a receiver means adding a second **Strategy** at a new address.

remove_strategy sets **allowed = false**, and **perform_refill** requires **allowed**. Disabling a strategy therefore also blocks the only path that returns its funds.

perform_refill transfers tokens into the bundle and lowers **average_external_equity** by the same amount. It has no outbound transfer, but it stamps **last_update_time** and writes **average_external_equity**, so the whitelist check bounds who can move the oracle.

remove_strategy leaves **allocation_bps** frozen at its old value, and **update_allocations** requires **allowed**, so the manager cannot shrink it before re-enabling. One disabled receiver held 47.93 USDC on Neutral-Earn-USDC-Bundle on 2026-09-23.

POTENTIAL BENEFIT

An operator recovers funds from a disabled strategy without first calling **enable_strategy**, which fails the 10000 bps cap once a replacement receiver holds the allocation.

RECOMMENDATION

Zero the allocation on removal, and make the refill path revocable instead of open.

RUST

```
strategy_account.allocation_bps = 0;   // in remove_strategy
pub winding_down: bool,                // one byte from Strategy::padding

require!(strategy_account.allowed || strategy_account.winding_down,
         ErrorCode::MustBeInWhitelist);
```

© 2026 Adevar Labs. All rights reserved.

Add a manager-only instruction that clears **winding_down** once the receiver has returned everything. Deleting the check outright extends the oracle write to all 41 Strategy PDAs.

FIX REVIEW NOTES

Acknowledged. Re-measured on 2026-10-02: five disabled receivers hold USDC, about 98 USDC in total, with the two largest holding about 50 and 48.

© 2026 Adevar Labs. All rights reserved.

E02: Manager fee withdrawal has no distribution gate, so it spends cash a started distribution already counted

LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/manager_withdraw.rs:L90-L120

RELEVANT SNIPPET

>_ manager_withdraw.rs — RUST

```rust
#[derive(Accounts)]
pub struct ManagerWithdraw<'info> {
    #[account(mut, address = bundle_account.manager @ ErrorCode::UnauthorizedManagerAction)]
    pub manager: Signer<'info>,

    #[account(mut, constraint = manager_token_account.owner == manager.key(), constraint = manager_token_account.mint == asset_address.key())]
    /// CHECK: manager token account
    pub manager_token_account: Account<'info, TokenAccount>,

    #[account(mut, constraint = asset_address.key() == bundle_account.asset_address.key())]
    /// CHECK: bundle asset mint
    pub asset_address: AccountInfo<'info>,

    #[account(mut,
        associated_token::mint = asset_address.key(),
        associated_token::authority = bundle_asset_authority.key()
    )]
    pub bundle_asset_account: Account<'info, TokenAccount>,

    #[account(mut)]
    pub bundle_account: Account<'info, Bundle>,

    #[account(mut, seeds = [ORACLE_SEED, bundle_account.key().as_ref()], bump)]
    pub oracle_data: Account<'info, OracleData>,

    #[account(mut, seeds = [BUNDLE_ASSET_AUTHORITY_SEED, bundle_account.key().as_ref()], bump)]
    pub bundle_asset_authority: SystemAccount<'info>,

    pub token_program: Program<'info, Token>,
}
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/start_distribution.rs:L30

DESCRIPTION

start_distribution snapshots **bundle_underlying_balance** into **distribution_base_amount**, and each **distribute_to_receivers** pays a share of that snapshot while debiting the live balance.

manager_withdraw and **manager_withdraw_with_split** debit the same balance without reading the round state. Neither **Accounts** struct includes **bundle_temp_data**, so the check is unavailable to them.

© 2026 Adevar Labs. All rights reserved.

A manager withdrawal between **start_distribution** and the final payout leaves the snapshot stale by the redeemed amount. At the 10000 bps allocation that 28 of the 29 live bundles use, the next payout asks for the full snapshot and **checked_sub** underflows.

POTENTIAL BENEFIT

Removes a stalled distribution round that today requires the keeper to call **end_distribution** and re-run the cycle.

RECOMMENDATION

Add **bundle_temp_data** to both manager withdrawal contexts and reject a withdrawal mid-round.

```rust
require!(!bundle_temp_data.is_distribution_started, ErrorCode::DistributionAlreadyStarted);
```

update_allocations has the same gap and takes the same guard.

FIX REVIEW NOTES

Acknowledged. The path is manager-triggered, and a stalled distribution round is recoverable with **end_distribution**.

© 2026 Adevar Labs. All rights reserved.

E03: The issuance fee's effective rate scales with pool size, so the configured rate is not the rate a depositor bears

LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/instant_mint_bundle_tokens.rs:L155-L165

RELEVANT SNIPPET

>_ instant_mint_bundle_tokens.rs RUST

```rust
153:     .ok_or(ErrorCode::MathError)?;
154:
155: let issuance_share_amounts = calculate_issuance_share_amounts(
156:     shares_minted,
157:     accounts.bundle_account.issuance_fee_bps,
158:     token_supply_before,
159: )
160: .map_err(|error_code| error!(error_code))?;
161: let tokens_to_mint = calculate_tokens_to_mint(
162:     issuance_share_amounts.issuance_net_shares,
163:     token_supply_before,
164:     pool_shares_before_credit,
165: )?;
166: require!(
167:     tokens_to_mint >= min_tokens_out,
```

DESCRIPTION

instant_mint_bundle_tokens credits the gross **shares_minted** to the pool and mints receipts against **issuance_net_shares**, so the fee shares stay in the pool with no tokens against them. That is the socialization the team pins in **issuance_fee_socialisation_strictly_increases_existing_holder_backing**.

The depositor then owns a slice of the pool holding their own fee, so the rate they bear is roughly **issuance_fee_bps * P / (P + N)** for a pool **P** and a mint **N**; the exact form, with the virtual offset, is in the recommendation. At 500 bps:

pool	deposit	fee taken	position after	really paid
10,000,000	10,000	500	9,500	500, 5.00%
1,000,000	1,000,000	50,000	974,346	25,654, 2.565%
1,000	1,000,000	50,000	999,894	106, 0.0106%

The configured rate is charged only on a deposit small against the pool. **issuance_fee_bps** is 0 on all 29 live bundles.

POTENTIAL BENEFIT

A manager setting the rate knows what it delivers, and can price the first mints into a young pool knowing they bear almost none of it.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

State the relationship where the fee is computed, and assert the effective rate in a test.

```rust
/// The rate a depositor bears is
///     issuance_fee_bps * (P + V) / (P + V + (1 - f) * N),
/// where P is pool backing shares before the credit, N the gross shares minted,
/// f = issuance_fee_bps / 10_000 and V = SHARE_TOKEN_VIRTUAL_OFFSET.
```

FIX REVIEW NOTES

Fixed. The issuance fee now backs no shares, so it lifts the whole bundle's price and a minter recaptures their bundle slice rather than their pool slice. A minter who owns most of the bundle still recaptures most of the fee, and that recapture is inherent to charging the fee into the bundle.

© 2026 Adevar Labs. All rights reserved.

E04: Instant mint bounds slippage on token count instead of value, so a tighter bound is cheaper to grief

LOCATION

bundle-sc/f015463/programs/ntbundle/src/instructions/instant_mint_bundle_tokens.rs:L166-L169

RELEVANT SNIPPET

>_ instant_mint_bundle_tokens.rs — RUST

```
164:     pool_shares_before_credit,
165: )?;
166: require!(
167:     tokens_to_mint >= min_tokens_out,
168:     ErrorCode::MinimumTokensOutNotMet
169: );
170: require!(tokens_to_mint != 0, ErrorCode::TokenAmountTooSmall);
171:
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/request_token_withdrawal.rs:L126

DESCRIPTION

instant_mint_bundle_tokens bounds slippage on the token count. **request_token_withdrawal** bounds the same feature on **min_shares_out**, the value unit.

An out-of-band **spl_token::burn** lowers **token_supply** without touching **tokenized_bundle_depositor.shares**. The quoted token count falls and the per-token claim rises by the same factor, so the caller's share value is unchanged and the guard reverts anyway.

Tripping it is expensive: the attacker burns **sigma * (token_supply + OFFSET)** tokens and forfeits the claim on them, roughly 10,000 USDC at a 1 percent tolerance on a 1,000,000 USDC pool, to cost the caller a network fee. The receipt token is a classic SPL mint, so no mint setting prevents a holder burning their own balance.

The team already reproduces this path in the test **uses the virtual offset and min_tokens_out after a real external burn**, which asserts the revert at **min_tokens_out = 100n** and a 95-token success at **95n**. Zero **TokenizedBundleDepositor** accounts exist on mainnet, and no script or SDK helper calls this path.

POTENTIAL BENEFIT

The mint guard fires only when the caller's share credit actually falls short, matching the redeem side.

RECOMMENDATION

Bound the mint on shares, as the redeem path already does. This changes the parameter from **min_tokens_out: u64** to **min_shares_out: u128**, which breaks the IDL.

RUST

```
require!(issuance_share_amounts.issuance_net_shares >= min_shares_out,
        ErrorCode::MinimumSharesOutNotMet);
```

© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Acknowledged. The path is tokenized-only, and blocking one mint costs the attacker the caller's slippage tolerance times the pool while gaining them nothing.

© 2026 Adevar Labs. All rights reserved.

E05: The withdrawal redemption schedule is checked only for non-zero values, so a transposed or expired schedule goes unnoticed

LOCATION

bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L217-L229

RELEVANT SNIPPET

>_ bundle.rs RUST

```rust
}

pub fn validate_withdrawal_redemption_schedule(
    cutoff_ts: i64,
    unlock_current_cycle_ts: i64,
    unlock_next_cycle_ts: i64,
) -> Result<()> {
    if cutoff_ts > 0 {
        require!(
            unlock_current_cycle_ts > 0 && unlock_next_cycle_ts > 0,
            ErrorCode::InvalidWithdrawalRedemptionSchedule
        );
    }
    Ok(())
}

pub fn calculate_withdrawal_cooldown_time(
```

OTHER LOCATIONS

- bundle-sc/f015463/programs/ntbundle/src/instructions/request_withdrawal.rs:L117-L125

DESCRIPTION

validate_withdrawal_redemption_schedule checks only that both unlock timestamps are non-zero, and only when **cutoff_ts > 0**. Nothing requires **unlock_next_cycle_ts** to follow **unlock_current_cycle_ts**, or either to fall after the cutoff, so a transposed pair pays users who miss the deadline before users who meet it.

The three consumers take **cooldown_end.max(policy_unlock)**, duplicated in **request_withdrawal**, **request_token_withdrawal** and **referrer_request_withdraw**. Once **unlock_next_cycle_ts** falls into the past, **max** always selects **cooldown_end** and the window stops applying, with no error and no event. The cooldown floor still holds in every case, so a bad schedule can only shorten the policy extension, never the cooldown itself.

No shipped script configures a schedule: **setWithdrawalRedemptionSchedule** is called nowhere in **migrations/** or **app/**, and every test constant is **0**. On chain, eight live bundles carry a schedule set outside those scripts (read 2026-10-02). All eight are ordered correctly, and Hyperithm-USDC-Bundle's expired on 2026-07-13, so the **policy_unlock** branch is live on the other seven.

POTENTIAL BENEFIT

Catches a transposed or expired schedule at the setter rather than at payout, and removes the only timing configuration in the program with no ordering rule.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Add the ordering and futurity rules **inside** the existing **cutoff_ts > 0** branch, and pass the clock in rather than reading it, so the helper stays callable from the unit tests in the same module.

RUST

```rust
pub fn validate_withdrawal_redemption_schedule(
    cutoff_ts: i64,
    unlock_current_cycle_ts: i64,
    unlock_next_cycle_ts: i64,
    now_ts: i64,
) -> Result<()> {
    if cutoff_ts > 0 {
        require!(
            unlock_current_cycle_ts > cutoff_ts,
            ErrorCode::InvalidWithdrawalRedemptionSchedule
        );
        require!(
            unlock_next_cycle_ts > unlock_current_cycle_ts,
            ErrorCode::InvalidWithdrawalRedemptionSchedule
        );
        require!(
            unlock_next_cycle_ts > now_ts,
            ErrorCode::InvalidWithdrawalRedemptionSchedule
        );
    }
    Ok(())
}
```

Both callers, **initialize_bundle** and **set_withdrawal_redemption_schedule**, already hold **Clock::get()?**. Placing the rules outside the branch would revert the disabled **0, 0, 0** configuration that the **initializeBundle** calls in **tests/ntbundle.ts** and **tests/user-fee-override.ts** pass today, and would block every new bundle that does not configure a schedule.

The duplicated **policy_unlock** block in the three request paths is worth extracting into one helper at the same time.

FIX REVIEW NOTES

Acknowledged. Only the manager sets schedules, and a bad one can only shorten the extra wait, never the cooldown. Seven live bundles run a schedule today, so review each one when it is set.

© 2026 Adevar Labs. All rights reserved.

E06: Strategy records no per-receiver debt, so one receiver's refill consumes another's repayment headroom

LOCATION

bundle-sc/f015463/programs/ntbundle/src/state/bundle.rs:L93-L101

RELEVANT SNIPPET

>_ bundle.rs RUST

```rust
}

#[account]
#[derive(InitSpace)]
pub struct Strategy {
    pub receiver_address: Pubkey, // address receiving funds associated with this strategy
    pub allocation_bps: u32,      // allocation in basis points
    pub allowed: bool,

    pub padding: [u8; 96], // 96 bytes padding
}

#[account]
```

DESCRIPTION

Strategy stores **receiver_address**, **allocation_bps** and **allowed** with no debt or principal field. **perform_refill** never reads **allocation_bps** and bounds **refill_amount** only by **checked_sub** against the single global **average_external_equity**. A receiver that repays more than it received leaves a second receiver unable to clear its own obligation, because the second **perform_refill** underflows and reverts, and the recorded NAV understates the fund until a keeper corrects it. The over-paying receiver sends real tokens and gains nothing.

Every live bundle runs a single receiver, for example Options Market Making at OKX through Copper at 95.74% and CTA Longshort Alpha at Binance through Ceffu at 99.82%, with the remainder in each being the bundle's own unallocated cash rather than a second receiver. **migrations/cli/initBundle.ts** matches, calling **addStrategy** exactly once at 9000 bps. With one receiver there is no second party to block. The code anticipates more: **distribute_to_receivers** carries **require!(bundle_account.allocated_receivers.len() < 20, TooManyAllocatedReceivers)**, and the Autopilot structure allocates across several trading firms. Apply this before whitelisting a second receiver. The likelier trigger is an operational error rather than intent, because the program gives a receiver no on-chain figure for its own outstanding principal.

POTENTIAL BENEFIT

Removes a reachable state in which a whitelisted receiver cannot return capital the bundle needs to pay withdrawals, and gives the program a figure against which an over-repayment can be detected rather than silently absorbed.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Record outstanding principal per receiver in **Strategy**, which already carries 96 bytes of padding, and bound **refill_amount** against it.

RUST

```rust
pub outstanding_principal: u64,
pub padding: [u8; 88],
```

FIX REVIEW NOTES

Acknowledged. Re-measured on 2026-10-02: each of the 29 live bundles runs one enabled receiver. This finding matters only once a bundle runs a second receiver.

© 2026 Adevar Labs. All rights reserved.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in Web3 audits.

The firm was built by a mix of experienced professionals from traditional enterprise security and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members whose publications have earned over 1,100 academic citations, and with the trust of projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the Web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is designed to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Finally, we examine external integrations with other Solana projects and verify that inputs and return values are handled properly.

© 2026 Adevar Labs. All rights reserved.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited, we may also include a detailed proof-of-concept exploit with code snippets and instructions on how the attack could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

© 2026 Adevar Labs. All rights reserved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, express or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time of the engagement.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 Adevar Labs. All rights reserved.